
UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

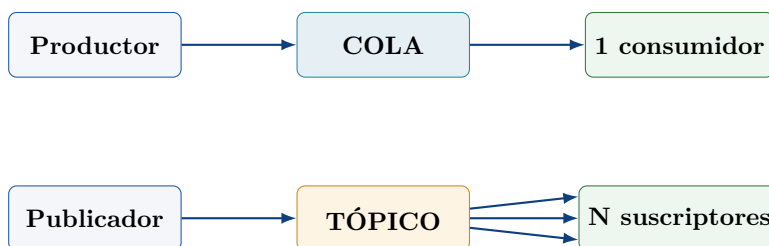
Carrera de Ingeniería de Software

MANUAL DE LABORATORIO

Mensajería asíncrona con JMS

Cola de mensajes y publicación/suscripción

Guía paso a paso con Java 21, Spring Boot, Apache ActiveMQ Artemis e IntelliJ IDEA — backend y frontend completos



Asignatura: Aplicaciones Distribuidas (ISR-701)

Nivel: Séptimo nivel – Ingeniería de Software

Período académico: 2026–2027 PPA

Autor: Prof. Gleiston C. Guerrero-Ulloa, M.Sc.

Versión: v1.0 – 30 de julio de 2026

Índice general

Cómo usar este manual	vii
I El problema y por qué necesita dos modelos de mensajería	1
1. El escenario: la mesa de ayuda de un proveedor de Internet	3
1.1. Enunciado	3
1.2. Primer intento: llamada síncrona directa	3
1.3. Segundo intento: ¿resolvemos todo con una cola?	4
1.4. Tercer intento: ¿resolvemos todo con un tópico?	4
1.5. Conclusión: se necesitan los dos	5
1.6. Arquitectura objetivo	5
II Fundamento teórico de JMS	7
2. Qué es JMS y qué no es	9
2.1. Una especificación, no un producto	9
2.2. El cambio de <code>javax</code> a <code>jakarta</code>	9
2.3. Por qué Artemis y no otro broker	10
3. Anatomía de la API JMS	11
3.1. La cadena de objetos	11
3.2. El mismo envío, en JMS puro y con Spring	11
4. Los dos dominios de mensajería	13
4.1. Dominio punto a punto: la cola	13
4.2. Dominio publicación/suscripción: el tópico	13
4.3. Tabla comparativa completa	14
5. El mensaje por dentro	15
5.1. Cabeceras	15
5.2. Propiedades	15
5.3. Cuerpo	16
6. Garantías de entrega	17
6.1. Modo de entrega: persistente o no persistente	17
6.2. Modos de acuse de recibo	17
6.3. Reintentos y cola de mensajes muertos	17
6.4. Idempotencia: la responsabilidad que JMS no asume	18

7. Suscripciones durables y selectores	19
7.1. El problema de la suscripción no durable	19
7.2. La suscripción durable	19
7.3. Selectores de mensajes	19
 III Preparación del entorno en Windows	 21
8. Requisitos e instalación	23
8.1. Paso 1.1 — Instalar el JDK 21	23
8.2. Paso 1.2 — Instalar IntelliJ IDEA	23
8.3. Paso 1.3 — Instalar Docker Desktop	24
9. Levantar el broker Apache ActiveMQ Artemis	25
9.1. Paso 2.1 — Crear la carpeta de trabajo	25
9.2. Paso 2.2 — Escribir el archivo <code>docker-compose.yml</code>	25
9.3. Paso 2.3 — Arrancar el broker	26
9.4. Paso 2.4 — Abrir la consola de administración	26
9.5. Comandos útiles del ciclo de vida	27
 IV Construcción de la aplicación	 29
10. Crear el proyecto en IntelliJ IDEA	31
10.1. Paso 3.1 — Nuevo proyecto Spring Boot	31
10.2. Paso 3.2 — Elegir la versión de Spring Boot y las dependencias	31
10.3. Paso 3.3 — Alternativa sin asistente	32
11. El archivo <code>pom.xml</code> explicado	33
11.1. Contenido completo	33
11.2. Qué significa cada bloque	34
12. Configuración de la aplicación	35
13. Estructura de paquetes y repaso del lenguaje	37
13.1. Estructura que vamos a crear	37
13.2. Los elementos del lenguaje que vamos a usar	37
13.2.1. Paquete e importaciones	37
13.2.2. <code>record</code> : datos inmutables sin ceremonia	38
13.2.3. <code>enum</code> : un conjunto cerrado de valores	38
13.2.4. Anotaciones	38
13.2.5. Inyección por constructor	39
13.2.6. Lambdas e interfaces funcionales	39

13.2.7. <code>var</code> , <code>final</code> y <code>static final</code>	40
13.2.8. Excepciones comprobadas y no comprobadas	40
14.El dominio: los objetos que viajarán	41
14.1. Paso 4.1 — <code>Prioridad.java</code>	41
14.2. Paso 4.2 — <code>Ticket.java</code> , el comando	41
14.3. Paso 4.3 — <code>EventoTicket.java</code> , el evento	41
14.4. Paso 4.4 — <code>SolicitudTicket.java</code> , la entrada del formulario	42
15.La configuración JMS: el corazón del laboratorio	45
15.1. Paso 5.1 — <code>JmsConfig.java</code>	45
15.2. Las cuatro decisiones de este archivo	47
15.2.1. Por qué dos <code>JmsTemplate</code> y no uno	47
15.2.2. Por qué el orden importa en las fábricas	47
15.2.3. Por qué se inyecta el <code>ObjectMapper</code>	47
15.2.4. Por qué <code>setTypeIdPropertyName</code>	48
16.El productor	49
16.1. Paso 6.1 — <code>TicketProductor.java</code>	49
17.Los consumidores de la cola: los tres técnicos	51
17.1. Paso 7.1 — <code>TecnicoConsumidor.java</code>	51
17.2. Qué está pasando por debajo	52
17.2.1. <code>@JmsListener</code> no es una llamada, es una suscripción	52
17.2.2. Por qué los contadores son <code>AtomicInteger</code>	52
17.2.3. Por qué el parámetro es directamente un <code>Ticket</code>	53
18.Los suscriptores del tópico	55
18.1. Paso 8.1 — <code>SuscriptoresEventos.java</code>	55
18.2. El detalle decisivo: <code>subscription</code>	56
19.La capa web: REST y eventos hacia el navegador	57
19.1. Paso 9.1 — <code>FilaTablero.java</code>	57
19.2. Paso 9.2 — <code>TableroSse.java</code>	57
19.3. Paso 9.3 — <code>TicketControlador.java</code>	58
19.4. Paso 9.4 — La clase de arranque	60
20.El frontend	61
20.1. Paso 10.1 — <code>index.html</code>	61
20.2. Paso 10.2 — <code>estilos.css</code>	62
20.3. Paso 10.3 — <code>app.js</code>	64

V	Ejecución, experimentos y diagnóstico	67
21.	Ejecutar la aplicación	69
21.1.	Paso 11.1 — Comprobar que el broker está vivo	69
21.2.	Paso 11.2 — Arrancar desde IntelliJ	69
21.3.	Paso 11.3 — Leer el arranque	69
21.4.	Paso 11.4 — Abrir el tablero	69
22.	Siete experimentos	71
22.1.	Experimento 1 — La cola reparte	71
22.2.	Experimento 2 — El tópico difunde	71
22.3.	Experimento 3 — Extensibilidad sin tocar al emisor	71
22.4.	Experimento 4 — La cola conserva; el tópico no	72
22.5.	Experimento 5 — La suscripción durable sí conserva	72
22.6.	Experimento 6 — Selectores	72
22.7.	Experimento 7 — Reintento y cola de mensajes muertos	72
23.	Errores frecuentes y cómo diagnosticarlos	75
23.1.	Método general de diagnóstico	75
23.2.	Catálogo de errores	75
24.	Extensiones y aplicación a los proyectos del período	77
24.1.	Cómo separar los componentes en procesos distintos	77
24.2.	Traslado a los cuatro dominios del período	77
24.3.	Relación con la calidad del producto	78
24.4.	Hacia dónde seguir	78
	Bibliografía	79

Cómo usar este manual

Este manual está escrito para alguien que nunca ha usado JMS y que quiere entender *por qué* cada línea está donde está, no solo copiarla. Por esa razón explica tanto el lenguaje (qué es un **record**, qué hace una anotación, por qué se inyecta por constructor) como la tecnología (qué es un *broker*, qué es una sesión JMS, por qué una suscripción durable necesita un identificador de cliente).

El recorrido tiene cinco partes. La Parte I plantea el problema y demuestra, con un razonamiento paso a paso, que hacen falta *los dos* modelos de mensajería y no uno solo. La Parte II desarrolla el fundamento teórico completo de JMS. La Parte III prepara el entorno de trabajo en Windows. La Parte IV construye la aplicación completa, archivo por archivo y línea por línea, en IntelliJ IDEA. La Parte V contiene siete experimentos que hacen visible la diferencia entre cola y tópico, más un catálogo de errores frecuentes.

Convenciones tipográficas

Lo que aparece en **tipografía monoespaciada** es texto que se escribe literalmente: un comando, un nombre de archivo, una clase o una propiedad. Los bloques numerados son código completo, listo para copiar; el número de línea sirve para que el texto explicativo pueda referirse a una línea concreta. Los comentarios dentro del código se escriben sin tildes de forma deliberada, porque algunos editores y consolas de Windows todavía manejan mal la codificación de caracteres en archivos fuente. Las cajas rojas señalan errores que romperán la aplicación; las amarillas, consejos prácticos; las verdes, definiciones formales; las grises, notas de contexto.

Si algo no funciona

Antes de reescribir código, ve al Capítulo 23. Recoge los quince fallos que aparecen con más frecuencia en el laboratorio, con el mensaje de error exacto que produce cada uno y su causa real. La mayoría de los problemas de un primer proyecto JMS no están en la lógica sino en tres sitios: el *broker* no está levantado, la propiedad `pubSubDomain` no coincide con el tipo de destino, o el conversor de mensajes no sabe serializar una fecha.

Parte I

El problema y por qué necesita dos modelos de mensajería

CAPÍTULO 1

El escenario: la mesa de ayuda de un proveedor de Internet

1.1 Enunciado

Un proveedor de servicios de Internet (ISP) opera una mesa de ayuda. El escenario corresponde al dominio del Proyecto Fin de Curso **ACC — Soporte Técnico ISP**, pero el razonamiento se traslada sin cambios a los otros tres dominios del período, como se muestra en el Capítulo 24.

Cuando un cliente reporta una avería —por ejemplo, “no tengo Internet desde las 14:00”— el sistema debe hacer dos cosas muy distintas entre sí:

1. **Asignar el ticket a un técnico.** Hay tres técnicos de turno. El ticket lo debe atender *exactamente uno*. Si lo atienden dos, se duplica el trabajo, se llama dos veces al cliente y se descuadra la métrica de productividad. Si no lo atiende ninguno, el cliente queda sin respuesta.
2. **Avisar a todos los sistemas interesados en el cambio de estado.** Cuando el ticket pasa a “EN_ATENCION” hay tres sistemas que deben enterarse *a la vez* y que no tienen nada que ver entre sí: el notificador que envía un correo o un mensaje al cliente, el monitor de acuerdos de nivel de servicio (SLA) que arranca el cronómetro de cumplimiento, y el registro de auditoría que deja constancia para reclamos posteriores.

Estos dos requisitos parecen del mismo tipo —“mandar un aviso”— pero su semántica es opuesta. El primero exige entrega *a uno*; el segundo exige entrega *a todos*. Esa oposición es exactamente la que separa los dos dominios de mensajería que define JMS, y es la razón por la que este laboratorio no es un ejercicio artificial: un sistema real de mesa de ayuda necesita los dos.

1.2 Primer intento: llamada síncrona directa

Supongamos que resolvemos todo con llamadas directas, del tipo HTTP síncrono o invocación de método. El controlador que recibe el ticket llamaría, uno tras otro, a los servicios implicados.

```
1 // ATENCION: este codigo ilustra el problema. NO es la solucion.
2 public void registrarTicket(Ticket t) {
3     Tecnico libre = servicioTecnicos.buscarTecnicoLibre(); // 1
4     servicioTecnicos.asignar(libre, t); // 2
5     servicioCorreo.notificarCliente(t); // 3
6     servicioSla.iniciarCronometro(t); // 4
7     servicioAuditoria.registrar(t); // 5
8 }
```

Listing 1.1: Solución ingenua: acoplamiento total (no usar)

Este código tiene cinco defectos graves, y conviene nombrarlos con precisión porque cada uno se corresponde con una propiedad concreta que la mensajería asíncrona sí ofrece.

Acoplamiento espacial.

El emisor conoce por nombre a los cuatro destinatarios. Si mañana el área de calidad pide un cuarto interesado —por ejemplo, un tablero de estadísticas— hay que *modificar y volver a desplegar* el código del emisor. Eso viola el principio de abierto/cerrado y convierte cada nuevo consumidor en un cambio de riesgo en el núcleo del sistema.

Acoplamiento temporal.

Los cinco pasos ocurren en el mismo instante. Si el servicio de correo está caído, la línea

3 lanza una excepción y el ticket *no se registra*, aunque registrar el ticket no dependa en absoluto del correo. Una funcionalidad secundaria tumba una funcionalidad crítica.

Acoplamiento de sincronización.

El hilo que atiende la petición HTTP del cliente queda bloqueado hasta que terminen los cinco pasos. Si el monitor de SLA tarda dos segundos, el cliente espera dos segundos de más. Con carga alta, el conjunto de hilos del servidor se agota.

Pérdida bajo indisponibilidad.

Si los tres técnicos están ocupados o el servicio de asignación está reiniciándose, la línea 1 falla y el ticket se pierde. No hay ningún sitio donde el trabajo quede *en espera*.

Ausencia de balanceo.

Nada garantiza que el trabajo se reparta. La lógica de “buscar técnico libre” hay que escribirla, mantenerla y hacerla segura frente a concurrencia; dos peticiones simultáneas pueden elegir al mismo técnico.

Las tres formas de desacoplamiento

La literatura clásica sobre publicación/suscripción identifica precisamente estas tres dimensiones de desacoplamiento como la aportación central del paradigma: desacoplamiento en el *espacio* (las partes no se conocen entre sí), en el *tiempo* (no necesitan estar activas simultáneamente) y en la *sincronización* (el emisor no se bloquea esperando al receptor) [4]. Todo lo que vamos a construir en este manual es, en el fondo, una manera de comprar esas tres propiedades a cambio de introducir un intermediario.

1.3 Segundo intento: ¿resolvemos todo con una cola?

Podríamos poner una cola entre el emisor y cada destinatario. Funciona bien para el primer requisito: los tres técnicos escuchan la *misma* cola y el intermediario reparte los tickets entre ellos, uno cada vez. Ese patrón tiene nombre propio en el catálogo de patrones de integración: *Competing Consumers*, consumidores en competencia [5].

Pero falla para el segundo requisito. Si los tres sistemas interesados en el cambio de estado escuchan la misma cola, cada evento llegaría solo a uno de ellos: el cliente recibiría el correo pero no se registraría la auditoría, o se registraría la auditoría pero no arrancaría el cronómetro de SLA. El resultado sería un sistema que pierde información de forma silenciosa y no determinista.

La alternativa sería crear *una cola por cada interesado*: `eventos.correo`, `eventos.sla`, `eventos.auditoria`. Entonces el emisor tendría que enviar el mismo mensaje tres veces, a tres destinos que debe conocer por nombre. Es decir, volvemos al acoplamiento espacial del primer intento: añadir un cuarto interesado obliga a modificar el emisor. Hemos ganado desacoplamiento temporal, pero no espacial.

1.4 Tercer intento: ¿resolvemos todo con un tópico?

Podríamos poner un tópico para todo. Funciona perfectamente para el segundo requisito: el emisor publica una vez, sin saber quién escucha, y los tres suscriptores reciben cada uno su propia copia. Añadir un cuarto interesado es dar de alta un nuevo suscriptor, sin tocar una línea del emisor.

Pero falla estrepitosamente para el primero. Si los tres técnicos se suscriben al mismo tópico, *los tres* reciben *todos* los tickets. Tres técnicos llamarían al mismo cliente por la misma avería. El tópico difunde; no reparte.

1.5 Conclusión: se necesitan los dos

El razonamiento anterior no es retórico: cada uno de los tres intentos falla por una razón estructural distinta, y ninguna combinación de un solo modelo resuelve ambos requisitos sin reintroducir acoplamiento. La solución correcta usa cada modelo donde su semántica corresponde.

	Asignar ticket a técnico	Avisar del cambio de estado
Semántica requerida	Exactamente un receptor	Todos los interesados
Modelo JMS correcto	Cola (punto a punto)	Tópico (publicación/suscripción)
Patrón de integración	Consumidores en competencia	Canal de publicación/suscripción
Qué aporta	Balanceo de carga y entrega única	Extensibilidad sin tocar al emisor
Qué pasa si no hay receptor	El mensaje espera en la cola	Se pierde, salvo suscripción durable
Si se añade un receptor	Aumenta el rendimiento del reparto	Recibe una copia adicional
Riesgo si se elige el otro	Trabajo triplicado	Información perdida de forma silenciosa

La regla de decisión, en una frase

Si el mensaje representa **una orden de trabajo** que alguien debe ejecutar, va a una **cola**. Si el mensaje representa **un hecho que ya ocurrió** y del que otros pueden querer enterarse, va a un **tópico**. “Atiende este ticket” es una orden. “El ticket cambió de estado” es un hecho. Esta distinción entre comando y evento es la que gobierna casi todas las decisiones de diseño de mensajería en sistemas reales [8].

1.6 Arquitectura objetivo

La Figura 1.1 muestra lo que vamos a construir. El navegador envía el ticket por HTTP; el controlador lo deposita en la cola; uno de los tres técnicos lo toma; al terminar, publica un evento en el tópico; los tres suscriptores reciben ese evento; y todo lo que ocurre se refleja en el navegador en tiempo real mediante un canal de eventos servidor-cliente.

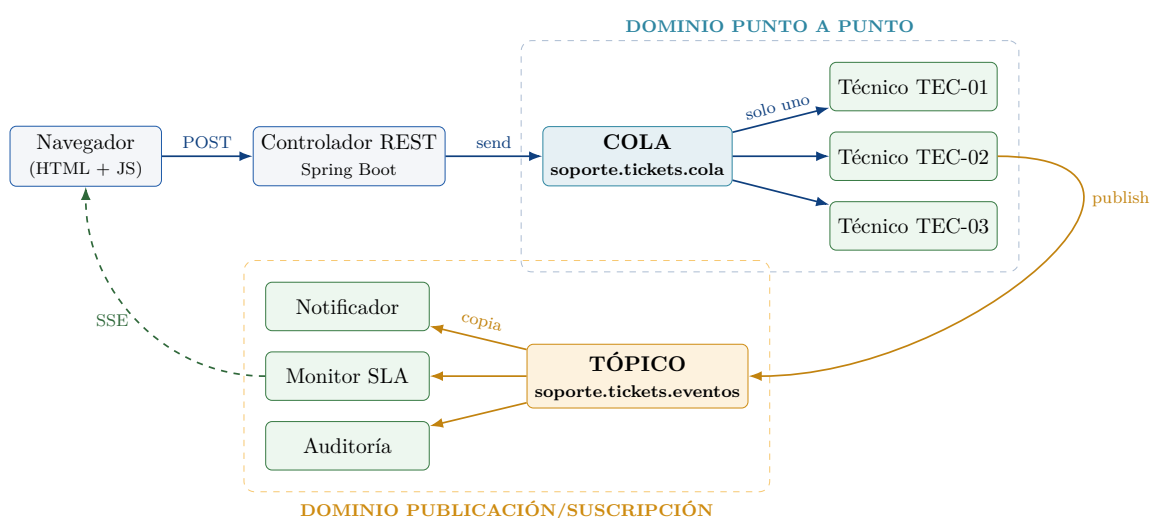


Figura 1.1: Arquitectura objetivo de la aplicación. La cola reparte cada ticket a un único técnico; el tópico entrega una copia del evento a cada suscriptor.

Por qué todo va en un solo proyecto

En un sistema real, cada técnico y cada suscriptor serían procesos independientes, quizá en máquinas distintas. Aquí conviven en una sola aplicación Spring Boot por una razón puramente pedagógica: así puedes ver los seis consumidores trabajando a la vez en una sola consola y en una sola pantalla, sin tener que arrancar seis terminales. La semántica de JMS es idéntica en ambos casos: al *broker* le da igual si sus clientes están en un proceso o en veinte. En el Capítulo 24 se explica cómo separarlos.

Parte II

Fundamento teórico de JMS

CAPÍTULO 2

Qué es JMS y qué no es

2.1 Una especificación, no un producto

JMS (*Java Message Service*) es una **especificación**: un contrato de interfaces que describe cómo un programa Java crea, envía, recibe y lee mensajes. No es un servidor, ni una librería que haga el trabajo, ni un producto que se pueda descargar y ejecutar.

Esta distinción es la primera fuente de confusión y conviene fijarla con una analogía. JDBC es una especificación para hablar con bases de datos relacionales; PostgreSQL es el producto que implementa el otro lado, y el *driver* de PostgreSQL es la pieza que traduce entre ambos. De igual modo, JMS es la especificación; Apache ActiveMQ Artemis es el producto —llamado **proveedor JMS** o, coloquialmente, **broker**— y el cliente Artemis es el *driver* que traduce.

Vocabulario mínimo

Especificación JMS: conjunto de interfaces del paquete `jakarta.jms` (antes `javax.jms`).

Proveedor JMS o broker: el servidor de mensajería que implementa esas interfaces y que realmente almacena y entrega los mensajes. Ejemplos: Apache ActiveMQ Artemis, IBM MQ, Oracle WebLogic JMS, Amazon SQS mediante su cliente JMS.

Cliente JMS: cualquier aplicación que produce o consume mensajes a través de esas interfaces.

Destino (*destination*): el buzón lógico donde se depositan los mensajes. Solo hay dos tipos: `Queue` y `Topic`.

Mensaje: la unidad de información que viaja. Tiene cabeceras, propiedades y cuerpo.

La consecuencia práctica de que JMS sea una especificación es enorme: el código que escribiremos en este manual funciona, sin cambios en la lógica de negocio, contra cualquier proveedor compatible. Cambiar de Artemis a IBM MQ es cambiar una dependencia y unas cuantas propiedades de conexión. Esa portabilidad es la razón de ser de la especificación.

2.2 El cambio de `javax` a `jakarta`

Durante veinte años los paquetes de JMS se llamaron `javax.jms`. Cuando Oracle donó Java EE a la Fundación Eclipse, la marca “Java” no pudo cederse, y todos los paquetes de la plataforma se renombraron a `jakarta`. Desde Jakarta EE 9 la interfaz es `jakarta.jms`, y la versión vigente de la especificación en la plataforma Jakarta EE 10 es **Jakarta Messaging 3.1** [3].

El error número uno al seguir tutoriales antiguos

Spring Boot 3.x usa exclusivamente `jakarta.jms`. Si copias un ejemplo de Internet escrito para Spring Boot 2.x, sus importaciones dirán `javax.jms.Message` y el compilador dará un error del tipo `package javax.jms does not exist`, o peor, compilará contra una dependencia antigua y fallará en tiempo de ejecución con `ClassNotFoundException`. La corrección es mecánica: cambiar `javax.jms` por `jakarta.jms` en todos los `import`. En IntelliJ IDEA basta con borrar la línea del `import` y pulsar `Alt+Enter` sobre el tipo subrayado para que ofrezca el paquete correcto.

2.3 Por qué Artemis y no otro broker

Para este laboratorio se usa Apache ActiveMQ Artemis por tres motivos concretos. Primero, implementa JMS de forma nativa y completa, incluidos tópicos y suscripciones durables, cosa que no ocurre con todos los intermediarios populares: Apache Kafka, por ejemplo, no expone una API JMS estándar y su modelo de consumo por grupos no coincide con la semántica de `Topic` de JMS. Segundo, se levanta con un único contenedor Docker y trae una consola web que permite *ver* las colas, los tópicos y los mensajes retenidos, lo cual es didácticamente decisivo. Tercero, es la línea de evolución oficial del proyecto ActiveMQ y está evaluada en la literatura arbitrada de comparación de intermediarios sobre la máquina virtual de Java, junto con Kafka, Pulsar y RocketMQ [2].

Sobre la elección de intermediario en un proyecto real

La elección no es trivial y depende del perfil de carga. Los estudios comparativos disponibles miden latencia, rendimiento, uso de CPU y comportamiento del recolector de basura, y coinciden en que no existe un ganador universal: Kafka domina en rendimiento bruto para flujos de altísimo volumen, mientras que los intermediarios orientados a colas ofrecen mejores garantías transaccionales por mensaje y una semántica de reintento más rica [2]. Para una mesa de ayuda, donde el volumen es moderado pero cada mensaje importa individualmente, un intermediario JMS es la elección natural.

CAPÍTULO 3

Anatomía de la API JMS

Aunque Spring Boot nos ahorrará escribir casi todo este código a mano, hay que conocerlo: cuando algo falle, el mensaje de error hablará de estos objetos, no de las abstracciones de Spring.

3.1 La cadena de objetos

Para enviar un solo mensaje en JMS puro hay que atravesar una cadena de cinco objetos. Cada uno tiene una responsabilidad y un ciclo de vida distinto.

Objeto	Qué es y para qué sirve	Coste
ConnectionFactory	Fábrica de conexiones. Guarda la dirección del <i>broker</i> y las credenciales. Es el único objeto que se obtiene “de fuera”: lo crea el proveedor o lo inyecta el contenedor. Es seguro compartirlo entre hilos.	Se crea una vez para toda la aplicación.
Connection	Conexión física al <i>broker</i> , sobre TCP. Es cara de crear porque implica apretón de manos de red y autenticación. Es segura entre hilos.	Cara. Se reutiliza.
Session	Contexto de trabajo de un solo hilo. Define si el trabajo es transaccional y qué modo de acuse de recibo se usa. No es segura entre hilos : cada hilo necesita la suya.	Barata, pero no compatible.
Destination	Referencia al buzón: Queue o Topic. Se obtiene de la sesión por nombre.	Trivial.
MessageProducer / MessageConsumer	El emisor y el receptor propiamente dichos. Se crean a partir de una sesión y de un destino.	Barato.

Session no es segura entre hilos

Esta es la regla que más código de producción rompe. Una **Session** pertenece a un hilo. Si dos hilos comparten una sesión, el comportamiento es indefinido: mensajes duplicados, mensajes perdidos o excepciones aparentemente aleatorias. Spring resuelve esto por nosotros creando una sesión por operación y almacenándolas en caché con **CachingConnectionFactory**, pero si algún día escribes JMS a mano, recuérdalo.

3.2 El mismo envío, en JMS puro y con Spring

Comparar ambos códigos deja clarísimo qué está haciendo Spring por nosotros. Primero, JMS puro:

```
1 // try-with-resources: Java cierra automáticamente los recursos al salir del bloque.
2 // Connection y Session implementan AutoCloseable, por eso pueden ir aquí.
3 try (Connection conexion = fabrica.createConnection("distapp", "distapp2026");
4      Session sesion = conexion.createSession(false, Session.AUTO_ACKNOWLEDGE)) {
5
6     // 1. Hay que arrancar la conexión de forma explícita para poder RECIBIR.
7     // Para enviar no es obligatorio, pero se suele hacer igual.
8     conexion.start();
9
10    // 2. Se obtiene el destino por su nombre. createQueue no crea nada en el
11    // broker si ya existe: devuelve una referencia al buzón.
12    Destination destino = sesion.createQueue("soporte.tickets.colas");
13
14    // 3. Se crea el productor asociado a ese destino.
15    MessageProducer productor = sesion.createProducer(destino);
```

```
16
17 // 4. Se construye el mensaje. TextMessage lleva una cadena en el cuerpo.
18 TextMessage mensaje = sesion.createTextMessage("{\"id\":\"TK-001\"}");
19
20 // 5. Las propiedades son metadatos definidos por la aplicacion.
21 mensaje.setStringProperty("prioridad", "ALTA");
22
23 // 6. Envio efectivo.
24 productor.send(mensaje);
25
26 } catch (JMSEException e) {
27 // JMSEException es una excepcion COMPROBADA: el compilador obliga a tratarla.
28 throw new IllegalStateException("Fallo al enviar el ticket", e);
29 }
```

Listing 3.1: Envío a una cola con la API JMS pura (referencia, no lo usaremos)

Y ahora exactamente lo mismo con Spring:

```
1 plantillaCola.convertAndSend("soporte.tickets.colas", ticket,
2     mensaje -> {
3         mensaje.setStringProperty("prioridad", "ALTA");
4         return mensaje;
5     });
```

Listing 3.2: El mismo envío con JmsTemplate

Spring ha absorbido seis responsabilidades: obtener la conexión, abrir y cerrar la sesión, resolver el destino, crear el productor, convertir el objeto Java a un mensaje JMS y traducir la `JMSEException` comprobada a una excepción no comprobada de su propia jerarquía. Lo que *no* ha absorbido, y por eso hay que entenderlo, es la decisión de si ese nombre designa una cola o un tópico.

CAPÍTULO 4

Los dos dominios de mensajería

4.1 Dominio punto a punto: la cola

En el dominio punto a punto el destino es una **Queue**. Sus reglas son las siguientes, y todas ellas importan.

1. Un mensaje depositado en una cola se entrega a **un único** consumidor, aunque haya diez escuchando.
2. Si hay varios consumidores, el *broker* los trata como consumidores en competencia y reparte los mensajes entre ellos. Añadir consumidores aumenta el rendimiento del sistema.
3. El mensaje **permanece en la cola** hasta que alguien lo consume y lo confirma, o hasta que caduca. Si no hay ningún consumidor activo, el mensaje espera.
4. Si el mensaje es persistente y el *broker* se reinicia, el mensaje sigue ahí.
5. El consumidor **confirma** la recepción. Si no confirma —por ejemplo, porque su código lanzó una excepción— el mensaje vuelve a la cola y se reintenta.

El punto 3 es el que resuelve el acoplamiento temporal, y el punto 5 el que da la garantía de no perder trabajo. El punto 2 es el que resuelve el balanceo sin escribir ni una línea de lógica de reparto.

4.2 Dominio publicación/suscripción: el tópico

En el dominio de publicación/suscripción el destino es un **Topic**. Sus reglas son casi opuestas.

1. Un mensaje publicado en un tópico se entrega a **todos** los suscriptores activos; cada uno recibe su propia copia.
2. El publicador no sabe cuántos suscriptores hay, ni quiénes son, ni si hay alguno. Publicar en un tópico sin suscriptores es una operación válida y silenciosa.
3. Por defecto, la suscripción es **no durable**: solo se reciben los mensajes publicados mientras el suscriptor está conectado. Lo que se publicó mientras estaba caído, se perdió para él.
4. Añadir un suscriptor no cambia nada en el publicador ni en los demás suscriptores.

El punto 3 sorprende siempre a quien viene de las colas y es el origen de la mitad de las preguntas del laboratorio: “publiqué diez eventos y mi suscriptor no recibió nada”. Casi siempre la causa es que el suscriptor arrancó después.

4.3 Tabla comparativa completa

Aspecto	Cola (punto a punto)	Tópico (pub/sub)
Interfaz JMS	Queue	Topic
Cardinalidad	Uno a uno	Uno a muchos
Receptores por mensaje	Exactamente uno	Todos los suscriptores
Efecto de añadir receptores	Reparte la carga	Multiplica las copias
Si no hay receptor activo	El mensaje espera	Se descarta (salvo durable)
Persistencia tras reinicio	Sí, si el mensaje es persistente	Solo con suscripción durable
Reintento tras fallo	Sí, vuelve a la cola	Sí, para ese suscriptor
Semántica típica	Comando, orden de trabajo	Evento, notificación de hecho
Acoplamiento del emisor	Conoce la cola destino	No conoce a nadie
Caso de nuestro laboratorio	Asignar ticket a un técnico	Avisar del cambio de estado
Propiedad de Spring	<code>pubSubDomain = false</code>	<code>pubSubDomain = true</code>

El mismo nombre no basta

En JMS, `soporte.eventos` como cola y `soporte.eventos` como tópico son **dos destinos distintos**. El nombre no determina el tipo; lo determina el método que se usa para resolverlo (`createQueue` frente a `createTopic`) o, en Spring, la propiedad `pubSubDomain`. Si el productor publica con `pubSubDomain = true` y el consumidor escucha con `pubSubDomain = false`, no habrá ningún error visible: simplemente el consumidor esperará eternamente en una cola vacía mientras los mensajes se difunden por un tópico que nadie escucha. Este es, con diferencia, el fallo más difícil de diagnosticar para un principiante, porque no produce excepción alguna.

CAPÍTULO 5

El mensaje por dentro

Un mensaje JMS tiene exactamente tres zonas. Saber cuál es cuál permite decidir dónde poner cada dato.

5.1 Cabeceras

Las cabeceras son campos predefinidos por la especificación. Casi todas las escribe el proveedor automáticamente al enviar.

Cabecera	La escribe	Significado
JMSMessageID	El proveedor	Identificador único del mensaje dentro del <i>broker</i> .
JMSTimestamp	El proveedor	Instante en que el mensaje se entregó al proveedor para su envío.
JMSDestination	El proveedor	Destino al que se envió.
JMSDeliveryMode	El productor	PERSISTENT o NON_PERSISTENT.
JMSPriority	El productor	Entero de 0 a 9. De 0 a 4 es prioridad normal; de 5 a 9, expedita. El valor por omisión es 4.
JMSExpiration	El proveedor	Instante de caducidad, calculado a partir del tiempo de vida indicado. Cero significa que no caduca.
JMSCorrelationID	La aplicación	Sirve para enlazar una respuesta con su petición.
JMSReplyTo	La aplicación	Destino donde el receptor debe contestar.
JMSRedelivered	El proveedor	<code>true</code> si este mensaje ya se intentó entregar antes. Muy útil para detectar reintentos.
JMSType	La aplicación	Etiqueta libre de tipo.

JMSPriority no es una promesa

La especificación pide que el proveedor *intente* entregar antes los mensajes de mayor prioridad, pero no lo garantiza de forma estricta. Si tu lógica de negocio depende de un orden exacto, no confíes en la prioridad: usa colas separadas o un selector.

5.2 Propiedades

Las propiedades son pares nombre-valor que define la aplicación. Admiten tipos primitivos y `String`. Su utilidad principal es doble: llevar metadatos que el consumidor puede leer sin deserializar el cuerpo, y servir de criterio a los *selectores* de mensajes.

```
1 // Al enviar
2 mensaje.setStringProperty("prioridad", "CRITICA");
3 mensaje.setIntProperty("intentos", 0);
4 mensaje.setBooleanProperty("clienteVip", true);
5
6 // Al recibir
7 String prioridad = mensaje.getStringProperty("prioridad");
8 boolean vip = mensaje.getBooleanProperty("clienteVip");
```

Listing 5.1: Escritura y lectura de propiedades

5.3 Cuerpo

El cuerpo lleva la carga útil. La especificación define cinco tipos, y elegir bien tiene consecuencias reales.

Tipo	Contenido	Cuándo usarlo
<code>TextMessage</code>	Una cadena de texto.	El más recomendable. JSON o XML dentro de una cadena. Interoperable con clientes no Java.
<code>ObjectMessage</code>	Un objeto Java serializado.	Solo entre aplicaciones Java idénticas. Frágil y con riesgo de seguridad.
<code>MapMessage</code>	Pares nombre-valor tipados.	Estructuras planas sencillas.
<code>BytesMessage</code>	Secuencia de bytes en bruto.	Archivos, imágenes, formatos binarios.
<code>StreamMessage</code>	Secuencia de valores primitivos.	Poco usado en la práctica.

Por qué evitaremos `ObjectMessage`

`ObjectMessage` usa la serialización nativa de Java. Eso implica tres problemas. El emisor y el receptor deben compartir exactamente la misma clase, con el mismo `serialVersionUID`: cambiar un campo rompe la compatibilidad y el mensaje deja de poder leerse. Ningún cliente escrito en Python, C# o JavaScript podrá interpretarlo. Y, sobre todo, deserializar objetos Java procedentes de una fuente externa es un vector de ejecución remota de código bien documentado, motivo por el cual los proveedores modernos exigen listas blancas de paquetes para permitirlo. En este manual usaremos `TextMessage` con JSON, que es la práctica estándar.

CAPÍTULO 6

Garantías de entrega

6.1 Modo de entrega: persistente o no persistente

PERSISTENT (valor por omisión).

El *broker* escribe el mensaje en almacenamiento estable *antes* de confirmar el envío al productor. Si el *broker* se apaga y vuelve, el mensaje sigue ahí. Es más lento porque implica una escritura a disco.

NON_PERSISTENT.

El mensaje vive solo en memoria. Es mucho más rápido, pero si el *broker* cae, el mensaje se pierde. Apropiado para telemetría de alta frecuencia donde perder una muestra es irrelevante.

Para nuestros tickets, la persistencia no es negociable: perder un ticket es perder un cliente. Para los eventos de tablero podría discutirse, pero mantendremos el valor por omisión.

6.2 Modos de acuse de recibo

El acuse de recibo (*acknowledgement*) es la señal con la que el consumidor le dice al *broker* “ya puedes borrar este mensaje”. Hasta que llega esa señal, el *broker* considera el mensaje pendiente y podrá reintentarlo.

Modo	Comportamiento
AUTO_ACKNOWLEDGE	La sesión confirma automáticamente cuando el método receptor termina <i>sin lanzar excepción</i> . Si lanza excepción, no confirma y el mensaje se reintentará. Es el valor por omisión y el que usaremos.
CLIENT_ACKNOWLEDGE	El consumidor decide cuándo confirmar llamando a <code>message.acknowledge()</code> . Da control fino, pero confirma <i>todos</i> los mensajes recibidos hasta ese punto en la sesión, no solo el actual.
DUPS_OK_ACKNOWLEDGE	Confirmación perezosa y agrupada. Más rápido, a costa de aceptar duplicados ocasionales.
Sesión transaccional	Los envíos y recepciones se agrupan; nada surte efecto hasta <code>commit()</code> , y <code>rollback()</code> lo deshace todo.

La consecuencia práctica de AUTO_ACKNOWLEDGE

Con este modo, **lanzar una excepción dentro del método consumidor equivale a rechazar el mensaje**. El *broker* lo devolverá a la cola y lo volverá a entregar, posiblemente a otro consumidor. Esto es exactamente lo que queremos para un ticket que no se pudo procesar por un fallo transitorio. Pero también significa que un error *permanente* —por ejemplo, un mensaje mal formado— entrará en un bucle infinito de reintentos, y de ahí la necesidad del apartado siguiente.

6.3 Reintentos y cola de mensajes muertos

Ningún proveedor reintentará indefinidamente. Tras un número configurable de intentos fallidos, el mensaje se desvía a una **cola de mensajes muertos** (*Dead Letter Queue*, DLQ), donde queda a disposición de un operador para su análisis. En Artemis, la DLQ por omisión se llama DLQ y el número de reintentos por omisión es diez.

Este mecanismo convierte un fallo silencioso en un fallo observable, y es un requisito de la característica de *fiabilidad* del modelo de calidad de producto de la norma ISO/IEC 25010, que en su edición vigente organiza la calidad en nueve características [6].

6.4 Idempotencia: la responsabilidad que JMS no asume

JMS garantiza entrega *al menos una vez* en el caso general. No garantiza *exactamente una vez*, porque eso es imposible de asegurar en un sistema distribuido sin coordinación adicional: si el consumidor procesa el mensaje y cae justo antes de confirmar, el mensaje se reentregará y se procesará dos veces.

La consecuencia es una regla de diseño que hay que interiorizar: **el consumidor debe ser idempotente**, es decir, procesar el mismo mensaje dos veces debe producir el mismo resultado que procesarlo una vez. La técnica habitual es llevar un registro de identificadores ya procesados y descartar los repetidos. El problema general de las garantías de entrega y de la duplicación en sistemas distribuidos está tratado en profundidad en la literatura de referencia de la asignatura [7], [9].

CAPÍTULO 7

Suscripciones durables y selectores

7.1 El problema de la suscripción no durable

Una suscripción no durable existe solo mientras el suscriptor está conectado. En cuanto se desconecta, el *broker* olvida que existía y descarta todo lo que se publique después. Para el notificador de correo esto puede ser aceptable: si el sistema de correo estuvo caído diez minutos, quizá no queramos inundar al cliente con avisos atrasados.

Para la auditoría, en cambio, es inaceptable. Un registro de auditoría con huecos no sirve como evidencia.

7.2 La suscripción durable

Una suscripción durable le pide al *broker* que *recuerde* al suscriptor por su nombre y que le guarde los mensajes publicados mientras estuvo ausente. Al reconectarse, el suscriptor recibe todo lo que se perdió.

Para que funcione hacen falta dos cosas, y omitir cualquiera de las dos produce un fallo:

1. Un **identificador de cliente** (*client ID*) único y estable en la conexión. Es la manera que tiene el *broker* de reconocer que el que vuelve es el mismo de antes.
2. Un **nombre de suscripción** estable. La pareja identificador de cliente más nombre de suscripción es la clave que identifica la suscripción durable.

Dos consecuencias incómodas de las suscripciones durables

Primera: el identificador de cliente debe ser único en todo el *broker*. Si arrancas dos instancias de la misma aplicación con el mismo identificador, la segunda fallará con un error del tipo `client-id already added`. Segunda: una suscripción durable **acumula mensajes indefinidamente** mientras el suscriptor no se conecte. Si un suscriptor durable queda olvidado, su cola interna crecerá hasta agotar el disco del *broker*. Las suscripciones durables abandonadas hay que borrarlas de forma explícita desde la consola de administración.

7.3 Selectores de mensajes

Un selector es un filtro que se aplica *en el broker*, del lado del servidor, antes de entregar nada. Se escribe con una sintaxis parecida a la cláusula `WHERE` de SQL92 y opera sobre las *propiedades* y las *cabeceras* del mensaje, nunca sobre el cuerpo.

```
1 // Solo tickets criticos
2 "prioridad = 'CRITICA'"
3
4 // Tickets criticos o altos, de clientes VIP
5 "prioridad IN ('CRITICA','ALTA') AND clienteVip = TRUE"
6
7 // Reentregas: mensajes que ya fallaron alguna vez
8 "JMSRedelivered = TRUE"
9
10 // Rango numerico
11 "intentos >= 3"
```

Listing 7.1: Ejemplos de selectores

Por qué el selector no puede mirar el cuerpo

El *broker* trata el cuerpo como una secuencia opaca de bytes; no sabe si dentro hay JSON, XML o una imagen. Las propiedades, en cambio, están indexadas y son legibles para él. Por eso, si un dato va a servir de criterio de filtrado, hay que **duplicarlo** como propiedad además de llevarlo en el cuerpo. Es una redundancia deliberada y correcta.

Parte III

Preparación del entorno en Windows

CAPÍTULO 8

Requisitos e instalación

Todo el manual asume Windows 10 u 11 y el intérprete de comandos clásico `cmd.exe`, no PowerShell. La razón es práctica: la sintaxis de escape de comillas y variables difiere entre ambos, y todos los comandos que siguen están probados para `cmd.exe`.

Cómo abrir `cmd.exe`

Pulsa **Windows + R**, escribe `cmd` y pulsa Intro. No uses la terminal de PowerShell ni la de Windows Terminal configurada con PowerShell por omisión. Dentro de IntelliJ IDEA, la terminal integrada se cambia en **File > Settings > Tools > Terminal > Shell path**, poniendo `cmd.exe`.

8.1 Paso 1.1 — Instalar el JDK 21

Java 21 es una versión de soporte extendido y es la que exige Spring Boot 3 como mínimo razonable. Descarga un JDK 21 de cualquier distribución (Eclipse Temurin, Oracle, Amazon Corretto o Azul Zulu) e instálalo con las opciones por omisión.

Verifica la instalación abriendo `cmd.exe` y escribiendo:

```
1 java -version
2 javac -version
```

Listing 8.1: Verificación del JDK

La salida debe mostrar una versión que empiece por 21. Si dice “no se reconoce como un comando interno o externo”, el instalador no añadió Java al `PATH`: reinstala marcando la opción correspondiente, o añade manualmente la carpeta `bin` del JDK a la variable de entorno `Path`.

JDK y JRE no son lo mismo

El JRE (*Java Runtime Environment*) solo ejecuta programas Java. El JDK (*Java Development Kit*) además los compila: incluye `javac`, el compilador. Si `java -version` funciona pero `javac -version` no, tienes un JRE y necesitas un JDK.

8.2 Paso 1.2 — Instalar IntelliJ IDEA

Descarga IntelliJ IDEA en su edición Community, que es gratuita y suficiente para todo este manual. Si eres estudiante puedes solicitar una licencia académica de la edición Ultimate, que añade herramientas visuales para Spring, pero no las necesitaremos.

Qué es un IDE y qué nos aporta aquí

Un IDE (entorno de desarrollo integrado) es un editor que además entiende el lenguaje. Las tres funciones que más usaremos son: completado de código con **Ctrl+Espacio**, importación automática de clases con **Alt+Enter** sobre un tipo desconocido, y navegación al origen de un símbolo con **Ctrl+clic**. La tercera es especialmente útil en este laboratorio: pulsando **Ctrl+clic** sobre `JmsTemplate` podrás leer directamente el código fuente de Spring y comprobar por ti mismo lo que este manual afirma.

8.3 Paso 1.3 — Instalar Docker Desktop

Docker nos permite levantar el *broker* Artemis sin instalarlo, sin configurarlo y sin ensuciar el sistema. Instala Docker Desktop y, cuando termine, verifica:

```
1 docker --version
2 docker compose version
```

Listing 8.2: Verificación de Docker

Docker Desktop debe estar *ejecutándose*

No basta con haberlo instalado. El icono de la ballena debe estar visible en la bandeja del sistema y en estado “Engine running”. Si no lo está, todos los comandos `docker` fallarán con un mensaje del tipo `error during connect: ... The system cannot find the file specified`. En Windows, Docker Desktop requiere además que WSL 2 esté habilitado; el propio instalador lo propone.

CAPÍTULO 9

Levantar el broker Apache ActiveMQ Artemis

9.1 Paso 2.1 — Crear la carpeta de trabajo

Abre `cmd.exe` y crea la estructura del laboratorio:

```
1 C:\> mkdir C:\labs\distapp-jms
2 C:\> cd C:\labs\distapp-jms
```

Listing 9.1: Creación de la carpeta de trabajo

9.2 Paso 2.2 — Escribir el archivo `docker-compose.yml`

Dentro de `C:\labs\distapp-jms` crea un archivo llamado `docker-compose.yml` con el contenido siguiente. Puedes crearlo desde el Bloc de notas o, más adelante, desde el propio IntelliJ.

```
1 services:
2   artemis:
3     image: apache/activemq-artemis:2.37.0
4     container_name: artemis-distapp
5     environment:
6       # Credenciales con las que se conectara nuestra aplicacion.
7       ARTEMIS_USER: distapp
8       ARTEMIS_PASSWORD: distapp2026
9       # Prohibimos el acceso anonimo: obliga a autenticarse siempre.
10      ANONYMOUS_LOGIN: "false"
11      # Hace que la consola web escuche en todas las interfaces del
12      # contenedor; sin esto solo escucharía en localhost DEL CONTENEDOR
13      # y no sería accesible desde Windows.
14      EXTRA_ARGS: "--http-host 0.0.0.0 --relax-jolokia"
15    ports:
16      # 61616: puerto del protocolo de mensajería (el que usa nuestra app).
17      - "61616:61616"
18      # 8161: consola web de administración.
19      - "8161:8161"
20    volumes:
21      # Persiste los datos del broker entre reinicios del contenedor.
22      - artemis-datos:/var/lib/artemis-instance
23
24 volumes:
25   artemis-datos:
```

Listing 9.2: `docker-compose.yml` — definición del broker

Explicación línea por línea

services:

Bloque que enumera los contenedores que forman la aplicación. Aquí solo hay uno.

image:

Imagen que se descargará de Docker Hub. Fijamos la versión de forma explícita en lugar de usar `latest`, porque `latest` cambia sin aviso y rompería la reproducibilidad del laboratorio.

container_name:

Nombre legible del contenedor, para poder referirse a él en comandos posteriores.

environment:

Variables de entorno que la imagen lee al arrancar para configurarse.

ports:

Mapeo `puertoWindows:puertoContenedor`. El puerto 61616 es por el que hablará nuestra aplicación Java; el 8161 es la consola web.

volumes:

Un volumen con nombre para que los mensajes persistentes sobrevivan a `docker compose down`.

Si tu versión de la imagen no existe

Si al arrancar aparece `manifest for apache/activemq-artemis:2.37.0 not found`, la etiqueta ha cambiado. Consulta las etiquetas disponibles en Docker Hub y sustituye por una versión 2.x existente. Las variables de entorno pueden variar entre versiones de la imagen; si la consola no abre, revisa la documentación de la imagen concreta que hayas descargado.

9.3 Paso 2.3 — Arrancar el broker

```
1 C:\labs\distapp-jms> docker compose up -d
```

Listing 9.3: Arranque del broker

La opción `-d` significa *detached*: el contenedor queda en segundo plano y devuelve el control de la consola. La primera vez tardará porque tiene que descargar la imagen.

Comprueba que está vivo:

```
1 C:\labs\distapp-jms> docker compose ps
2 C:\labs\distapp-jms> docker compose logs -f artemis
```

Listing 9.4: Comprobación del estado

En los registros debes ver una línea que indique que el servidor está activo y aceptando conexiones. Pulsa `Ctrl+C` para dejar de seguir los registros; eso *no* detiene el contenedor.

9.4 Paso 2.4 — Abrir la consola de administración

Abre el navegador en `http://localhost:8161/console` y entra con el usuario `distapp` y la contraseña `distapp2026`.

Esta consola es la herramienta de diagnóstico más valiosa del laboratorio. Desde ella podrás, en la sección de direcciones y colas, ver cuántos mensajes hay pendientes, cuántos se han entregado, cuántos consumidores están conectados y qué contiene cada mensaje retenido. Cuando algo no funcione, la primera pregunta siempre será: ¿el mensaje llegó al *broker*? La consola responde a esa pregunta en cinco segundos.

Direcciones, colas y tópicos en Artemis

Artemis tiene un modelo interno ligeramente distinto al de JMS clásico y conviene conocerlo para no desorientarse en la consola. Internamente todo son *direcciones* (*addresses*), y cada dirección tiene un modo de enrutamiento: `ANYCAST` o `MULTICAST`. Una `Queue` de JMS es una dirección `ANYCAST` con una cola asociada. Un `Topic` de JMS es una dirección `MULTICAST`: cada suscriptor genera dentro de ella su propia cola interna, y por eso cada uno recibe su copia. Cuando en la consola veas que tu tópico tiene tres colas hijas con nombres largos y aparentemente aleatorios, eso es exactamente lo que estás viendo: los tres suscriptores no durables.

9.5 Comandos útiles del ciclo de vida

Comando	Efecto
<code>docker compose up -d</code>	Arranca el <i>broker</i> en segundo plano.
<code>docker compose stop</code>	Lo detiene conservando los datos.
<code>docker compose start</code>	Lo vuelve a arrancar.
<code>docker compose down</code>	Lo detiene y borra el contenedor; el volumen se conserva.
<code>docker compose down -v</code>	Lo detiene y borra también el volumen: se pierden todos los mensajes.
<code>docker compose logs -f artemis</code>	Muestra los registros en vivo.

Parte IV

Construcción de la aplicación

CAPÍTULO 10

Crear el proyecto en IntelliJ IDEA

10.1 Paso 3.1 — Nuevo proyecto Spring Boot

En IntelliJ IDEA, elige **File > New > Project**. En la lista de la izquierda selecciona **Spring Boot** (en la edición Community aparece como *Spring Initializr* tras instalar el complemento correspondiente; si no está disponible, usa la alternativa del apartado 10.3).

Rellena la primera pantalla exactamente así:

Campo	Valor y por qué
Name	mesa-ayuda-jms
Location	C:\labs\distapp-jms
Language	Java
Type	Maven — gestor de dependencias y de compilación.
Group	ec.edu.uteq.distapp — identificador de la organización, en notación de dominio invertido.
Artifact	mesa-ayuda-jms — nombre del artefacto compilado.
Package name	ec.edu.uteq.distapp.jms — paquete raíz del código.
JDK	21
Java	21
Packaging	Jar

Qué es un paquete y por qué se escribe al revés

Un paquete en Java es una carpeta lógica que agrupa clases y evita colisiones de nombres. Por convención se usa el dominio de la organización invertido: el dominio `uteq.edu.ec` da el paquete `ec.edu.uteq`. Así, si dos librerías distintas definen una clase llamada `Ticket`, sus nombres completos serán distintos y no habrá ambigüedad. La estructura de carpetas en disco debe coincidir con la del paquete: el paquete `ec.edu.uteq.distapp.jms` vive en `src/main/java/ec/edu/uteq/distapp/jms`.

10.2 Paso 3.2 — Elegir la versión de Spring Boot y las dependencias

En la segunda pantalla, selecciona la versión **más reciente de la rama 3.x** (por ejemplo 3.4.x).

No selecciones una versión 4.x

Este manual está escrito y verificado para Spring Boot 3.x. Las versiones 4.x introducen cambios en la configuración automática de Jackson y en algunas propiedades de JMS que harían que varios pasos de este manual no funcionasen tal cual están escritos. Si el asistente solo te ofrece versiones 4.x, escribe la versión 3.x a mano en el `pom.xml` como se indica en el Capítulo 11.

Marca estas dependencias:

Dependencia	Para qué la necesitamos
Spring Web	Servidor HTTP embebido (Tomcat), controladores REST y servicio de archivos estáticos para el frontend.
Spring for Apache ActiveMQ Artemis	Cliente JMS de Artemis más la configuración automática de Spring: <code>ConnectionFactory</code> , <code>JmsTemplate</code> y contenedores de escucha.
Validation	Anotaciones de validación como <code>@NotBlank</code> para comprobar los datos que llegan del formulario.
Spring Boot DevTools	Reinicio automático al guardar cambios. Opcional pero muy cómodo.

Pulsa **Create**. IntelliJ generará el proyecto y Maven empezará a descargar dependencias; la primera vez puede tardar varios minutos.

10.3 Paso 3.3 — Alternativa sin asistente

Si tu edición de IntelliJ no ofrece el asistente, ve a <https://start.spring.io> en el navegador, rellena los mismos campos, marca las mismas dependencias, pulsa **Generate**, descomprime el `.zip` en `C:\labs\distapp-jms` y ábrelo en IntelliJ con **File > Open**, seleccionando la carpeta que contiene el `pom.xml`.

CAPÍTULO 11

El archivo pom.xml explicado

11.1 Contenido completo

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5                             https://maven.apache.org/xsd/maven-4.0.0.xsd">
6
7     <modelVersion>4.0.0</modelVersion>
8
9     <!-- El POM padre aporta las versiones coherentes de TODAS las
10         dependencias de Spring Boot. Gracias a el, mas abajo no hace
11         falta escribir <version> en cada dependencia. -->
12     <parent>
13         <groupId>org.springframework.boot</groupId>
14         <artifactId>spring-boot-starter-parent</artifactId>
15         <version>3.4.5</version>
16         <relativePath/>
17     </parent>
18
19     <groupId>ec.edu.uteq.distapp</groupId>
20     <artifactId>mesa-ayuda-jms</artifactId>
21     <version>1.0.0</version>
22     <name>mesa-ayuda-jms</name>
23     <description>Mesa de ayuda ISP con JMS: cola y publicacion/suscripcion</
24         description>
25
26     <properties>
27         <java.version>21</java.version>
28         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
29     </properties>
30
31     <dependencies>
32
33         <!-- Servidor HTTP embebido, controladores REST, archivos estaticos
34             y Jackson para JSON. -->
35         <dependency>
36             <groupId>org.springframework.boot</groupId>
37             <artifactId>spring-boot-starter-web</artifactId>
38         </dependency>
39
40         <!-- Cliente JMS de Artemis y autoconfiguracion JMS de Spring.
41             Esta es LA dependencia clave de este laboratorio. -->
42         <dependency>
43             <groupId>org.springframework.boot</groupId>
44             <artifactId>spring-boot-starter-artemis</artifactId>
45         </dependency>
46
47         <!-- Anotaciones de validacion: @NotBlank, @Size, @NotNull. -->
48         <dependency>
49             <groupId>org.springframework.boot</groupId>
50             <artifactId>spring-boot-starter-validation</artifactId>
51         </dependency>
52
53         <!-- Reinicio automatico al guardar. Opcional. -->
54         <dependency>
55             <groupId>org.springframework.boot</groupId>
56             <artifactId>spring-boot-devtools</artifactId>
57             <scope>runtime</scope>
58             <optional>true</optional>
59         </dependency>
60
61         <!-- JUnit 5 y utilidades de prueba. Solo en el ambito de test. -->
62         <dependency>
63             <groupId>org.springframework.boot</groupId>
64             <artifactId>spring-boot-starter-test</artifactId>
```

```
64         <scope>test</scope>
65     </dependency>
66
67 </dependencies>
68
69 <build>
70     <plugins>
71         <!-- Permite ejecutar la aplicacion con "mvn spring-boot:run" y
72              empaquetarla como jar ejecutable con "mvn package". -->
73         <plugin>
74             <groupId>org.springframework.boot</groupId>
75             <artifactId>spring-boot-maven-plugin</artifactId>
76         </plugin>
77     </plugins>
78 </build>
79
80 </project>
```

Listing 11.1: pom.xml completo

11.2 Qué significa cada bloque

<parent>

Maven soporta herencia de configuración. Al declarar como padre el **spring-boot-starter-parent**, heredamos una lista enorme de versiones compatibles entre sí, la configuración del compilador (incluida la opción **-parameters**, que resultará importante) y la codificación UTF-8. Por eso las dependencias de abajo no llevan **<version>**: la hereda del padre.

<groupId>, <artifactId>, <version>

Las tres coordenadas que identifican de forma única un artefacto en el repositorio de Maven.

<properties>

Variables del proyecto. **java.version** le dice al padre qué versión del lenguaje compilar.

starter

Un *starter* no es una librería, sino un paquete de dependencias coherentes. **spring-boot-starter-web** arrastra Tomcat, Spring MVC, Jackson y una veintena de artefactos más. Es una comodidad, no una capa mágica.

<scope>

Delimita cuándo está disponible la dependencia. **test** significa que solo se usa al compilar y ejecutar pruebas, y que no acaba en el **jar** final. **runtime** significa que no hace falta para compilar, solo para ejecutar.

spring-boot-maven-plugin

Empaqueta la aplicación como un **jar** ejecutable que incluye el servidor web dentro. Es lo que permite ejecutar el sistema con un simple **java -jar**.

Recargar Maven tras editar el pom.xml

Cada vez que modifiques el **pom.xml**, IntelliJ mostrará un icono flotante de Maven en la esquina superior derecha del editor. Púlsalo, o usa **Ctrl+Shift+0**, para que descargue las dependencias nuevas. Si no lo haces, seguirás viendo errores de compilación por clases que “no existen” aunque la dependencia ya esté declarada.

CAPÍTULO 12

Configuración de la aplicación

Abre `src/main/resources/application.properties` y sustituye su contenido por lo siguiente.

```
1 # --- Identidad de la aplicacion -----
2 spring.application.name=mesa-ayuda-jms
3 server.port=8080
4
5 # --- Conexion al broker Artemis -----
6 # mode=native significa: conectarse a un broker EXTERNO ya en marcha.
7 # La alternativa, embedded, arrancaria un broker dentro del proceso Java;
8 # no la usamos porque queremos ver el broker de verdad en la consola web.
9 spring.artemis.mode=native
10 spring.artemis.broker-url=tcp://localhost:61616
11 spring.artemis.user=distapp
12 spring.artemis.password=distapp2026
13
14 # --- Cache de conexiones JMS -----
15 # Reutiliza conexiones y sesiones en lugar de abrirlas y cerrarlas en
16 # cada envio. Sin esto, el rendimiento cae en picado.
17 spring.jms.cache.enabled=true
18 spring.jms.cache.session-cache-size=10
19
20 # --- Registro de lo que ocurre -----
21 # Nivel INFO para nuestro codigo; asi veremos los mensajes del laboratorio.
22 logging.level.ec.edu.uteq.distapp.jms=INFO
23 # Formato compacto y con hora, para leer bien la consola.
24 logging.pattern.console=%d{HH:mm:ss.SSS} %-5level [%15.15thread] %-42logger{0} : %msg%
    n
```

Listing 12.1: application.properties

La propiedad que deliberadamente NO ponemos

Existe una propiedad global `spring.jms.pub-sub-domain` que fija el dominio para *toda* la aplicación. **No la usaremos**, y es importante entender por qué: nuestra aplicación necesita los dos dominios a la vez. Si la pusiéramos a `true`, las colas dejarían de funcionar; si a `false`, dejarían de funcionar los tópicos. En su lugar configuraremos el dominio *por cada componente*, en el Capítulo 15. Buena parte de los tutoriales de Internet usan esta propiedad global precisamente porque solo demuestran uno de los dos modelos.

El significado del patrón de registro

La cadena de `logging.pattern.console` es una plantilla. `%d{HH:mm:ss.SSS}` imprime la hora con milisegundos, que necesitaremos para ver el orden real de los eventos. `%-5level` imprime el nivel alineado a cinco caracteres. `%15.15thread` imprime el nombre del hilo recortado a quince caracteres: será fundamental, porque nos permitirá comprobar que cada consumidor trabaja en su propio hilo. `%-42logger{0}` imprime el nombre simple de la clase que emite el registro. `%msg%n` imprime el mensaje y un salto de línea.

CAPÍTULO 13

Estructura de paquetes y repaso del lenguaje

13.1 Estructura que vamos a crear

```
1 mesa-ayuda-jms/  
2 |-- docker-compose.yml  
3 |-- pom.xml  
4 '-- src/  
5     '-- main/  
6         |-- java/ec/edu/uteq/distapp/jms/  
7             |-- MesaAyudaJmsApplication.java      (arranque)  
8             |-- config/  
9                 |-- JmsConfig.java                (destinos, plantillas, fabricas)  
10                |-- dominio/  
11                    |-- Prioridad.java             (enum)  
12                    |-- Ticket.java                (record: el comando)  
13                    |-- EventoTicket.java          (record: el evento)  
14                    |-- SolicitudTicket.java        (record: entrada del formulario)  
15                |-- mensajeria/  
16                    |-- TicketProductor.java       (envia a cola y publica en topico)  
17                    |-- TecnicoConsumidor.java      (3 consumidores de la COLA)  
18                    |-- SuscriptoresEventos.java    (3 suscriptores del TOPICO)  
19                '-- web/  
20                    |-- TicketControlador.java      (REST + SSE)  
21                    |-- TableroSse.java              (difusion al navegador)  
22                    |-- FilaTablero.java            (record de la fila del tablero)  
23            '-- resources/  
24                |-- application.properties  
25                '-- static/  
26                    |-- index.html  
27                    |-- estilos.css  
28                    '-- app.js
```

Listing 13.1: Estructura completa del proyecto

Cómo crear paquetes y clases en IntelliJ

En el panel de proyecto, clic derecho sobre `ec.edu.uteq.distapp.jms` y luego **New > Package** para los paquetes. Para las clases, clic derecho sobre el paquete y **New > Java Class**; en el cuadro de diálogo puedes elegir **Class**, **Record**, **Enum** o **Interface**. Para los archivos del frontend, clic derecho sobre `static` y **New > File**, escribiendo el nombre con su extensión.

13.2 Los elementos del lenguaje que vamos a usar

Antes de escribir código, conviene tener claros los ocho elementos de Java que aparecerán una y otra vez.

13.2.1 Paquete e importaciones

```
1 package ec.edu.uteq.distapp.jms.dominio;    // A que paquete pertenece ESTA clase.  
2  
3 import java.time.Instant;                   // Traer una clase de otro paquete.  
4 import java.util.List;
```

Listing 13.2: Encabezado de cualquier archivo Java

La línea `package` debe ser la primera instrucción del archivo y debe coincidir con la ruta de carpetas. Los `import` evitan tener que escribir el nombre completo (`java.time.Instant`) cada

vez.

13.2.2 record: datos inmutables sin ceremonia

Un **record** es una clase cuyo único propósito es transportar datos. El compilador genera automáticamente el constructor, los métodos de acceso, `equals`, `hashCode` y `toString`.

```

1 // Version moderna: una línea.
2 public record Punto(int x, int y) { }
3
4 // Lo que el compilador genera por debajo, aproximadamente:
5 public final class Punto {
6     private final int x;
7     private final int y;
8     public Punto(int x, int y) { this.x = x; this.y = y; }
9     public int x() { return x; }           // OJO: se llama x(), no getX()
10    public int y() { return y; }
11    // mas equals, hashCode y toString
12 }

```

Listing 13.3: Un record y su equivalente en clase tradicional

Los métodos de acceso de un record no llevan get

Si el componente se llama `prioridad`, el método de acceso es `prioridad()`, no `getPrioridad()`. Escribir `ticket.getPrioridad()` produce el error `cannot find symbol`. Los **record** son además **inmutables**: no existen métodos `set`. Una vez creado, un **record** no cambia. Esa inmutabilidad es exactamente lo que se quiere en un mensaje, porque un mensaje describe algo que ya ocurrió y no debe poder alterarse en tránsito.

13.2.3 enum: un conjunto cerrado de valores

```

1 public enum Prioridad {
2     BAJA(1), MEDIA(2), ALTA(3), CRITICA(4); // Las cuatro únicas instancias.
3
4     private final int peso;                // Cada instancia lleva un dato.
5
6     Prioridad(int peso) { this.peso = peso; } // Constructor, siempre privado.
7
8     public int peso() { return peso; }
9 }

```

Listing 13.4: Un enum con datos asociados

La ventaja frente a usar cadenas sueltas es que el compilador impide escribir `Prioridad.URGENTE` si esa constante no existe, mientras que la cadena `"URGENTE"` compilaría y fallaría en ejecución.

13.2.4 Anotaciones

Una anotación es un metadato que se adjunta a una clase, método o parámetro. Por sí sola no ejecuta nada: es una marca que otro programa lee.

Anotación	Qué le dice a Spring
@SpringBootApplication	Esta es la clase de arranque; busca componentes desde aquí hacia abajo.
@Configuration	Esta clase define objetos gestionados (<i>beans</i>).
@Bean	El objeto que devuelve este método debe quedar bajo gestión de Spring.
@Component, @Service	Crea una instancia única de esta clase y guárdala. @Service es @Component con intención semántica.
@RestController	Esta clase atiende peticiones HTTP y devuelve datos, no vistas.
@PostMapping, @GetMapping	Asocia este método a una ruta y a un verbo HTTP.
@RequestBody	Convierte el JSON del cuerpo de la petición en este objeto Java.
@JmsListener	Clave. Suscribe este método a un destino JMS; se invocará solo cada vez que llegue un mensaje.
@Qualifier	Cuando hay varios objetos del mismo tipo, indica cuál se quiere.
@Primary	Marca cuál es el preferido si nadie especifica.

13.2.5 Inyección por constructor

Spring crea las instancias y las entrega a quien las necesite. La forma correcta de recibirlas es por constructor.

```

1  @Service
2  public class TicketProductor {
3
4      // final: se asigna una sola vez, en el constructor, y no cambia mas.
5      private final JmsTemplate plantillaCola;
6
7      // Si la clase tiene UN SOLO constructor, Spring lo usa automaticamente
8      // sin necesidad de @Autowired.
9      public TicketProductor(JmsTemplate plantillaCola) {
10         this.plantillaCola = plantillaCola;
11     }
12 }

```

Listing 13.5: Inyección por constructor

Por qué constructor y no @Autowired sobre el campo

Con inyección por constructor, el objeto nunca puede existir en un estado incompleto, los campos pueden ser `final` y la clase se puede instanciar en una prueba unitaria sin levantar Spring. La inyección sobre el campo con `@Autowired` no permite nada de eso y está desaconsejada desde hace años.

13.2.6 Lambdas e interfaces funcionales

Una lambda es una función escrita en línea. Se puede usar allí donde se espera una interfaz con un único método abstracto.

```

1  // Forma antigua, con clase anonima:
2  plantilla.convertAndSend(destino, ticket, new MessagePostProcessor() {
3      @Override
4      public Message postProcessMessage(Message mensaje) throws JMSEException {
5          mensaje.setStringProperty("prioridad", "ALTA");
6          return mensaje;
7      }
8  });
9
10 // Forma moderna, con lambda: hace exactamente lo mismo.
11 plantilla.convertAndSend(destino, ticket, mensaje -> {
12     mensaje.setStringProperty("prioridad", "ALTA");
13     return mensaje;
14 });

```

```
14 };
```

Listing 13.6: La misma lógica con clase anónima y con lambda

13.2.7 `var`, `final` y `static final`

`var` deja que el compilador deduzca el tipo de una variable local; el tipo sigue siendo estricto, solo se ahorra escribirlo. `final` sobre un campo significa que solo se asigna una vez. `static final` define una constante compartida por todas las instancias, que es como declararemos los nombres de los destinos.

13.2.8 Excepciones comprobadas y no comprobadas

`JMSException` es *comprobada*: el compilador obliga a capturarla o a declararla. Spring la convierte en `JmsException`, que es *no comprobada* y no obliga a nada. Esa conversión es una de las razones por las que el código con `JmsTemplate` es tan corto.

CAPÍTULO 14

El dominio: los objetos que viajarán

Vamos a escribir cuatro tipos. Tres viajan por la red como mensajes; el cuarto solo entra desde el formulario.

14.1 Paso 4.1 — Prioridad.java

Crea el paquete dominio y dentro un enum llamado Prioridad.

```
1 package ec.edu.uteq.distapp.jms.dominio;
2
3 /**
4  * Nivel de urgencia de un ticket de soporte.
5  * El campo minutosSla indica en cuantos minutos debe atenderse
6  * segun el acuerdo de nivel de servicio del ISP.
7  */
8 public enum Prioridad {
9
10     BAJA(480),
11     MEDIA(240),
12     ALTA(60),
13     CRITICA(15);
14
15     private final int minutosSla;
16
17     Prioridad(int minutosSla) {
18         this.minutosSla = minutosSla;
19     }
20
21     public int minutosSla() {
22         return minutosSla;
23     }
24 }
```

Listing 14.1: dominio/Prioridad.java

14.2 Paso 4.2 — Ticket.java, el comando

```
1 package ec.edu.uteq.distapp.jms.dominio;
2
3 import java.time.Instant;
4
5 /**
6  * Orden de trabajo que viaja por la COLA.
7  * Semanticamente es un COMANDO: "atiende este ticket".
8  * Por eso va a una cola y lo debe recibir un unico tecnico.
9  */
10 public record Ticket(
11     String id,
12     String cliente,
13     String servicio,
14     Prioridad prioridad,
15     String descripcion,
16     Instant creadoEn) {
17 }
```

Listing 14.2: dominio/Ticket.java

14.3 Paso 4.3 — EventoTicket.java, el evento

```
1 package ec.edu.uteq.distapp.jms.dominio;
```

```

2
3 import java.time.Instant;
4
5 /**
6  * Hecho consumado que viaja por el TOPICO.
7  * Semanticamente es un EVENTO: "el ticket cambio de estado".
8  * Por eso va a un topico y lo reciben TODOS los interesados.
9  *
10 * Notese el tiempo verbal de los campos: describe algo que YA paso.
11 */
12 public record EventoTicket(
13     String ticketId,
14     String cliente,
15     Prioridad prioridad,
16     String estadoAnterior,
17     String estadoNuevo,
18     String tecnico,
19     Instant ocurridoEn) {
20 }

```

Listing 14.3: dominio/EventoTicket.java

Por qué el evento no lleva el ticket entero

Un evento debe llevar lo que sus consumidores necesitan, no todo lo que el emisor tiene. El monitor de SLA necesita la prioridad y el instante; el notificador necesita el cliente; la auditoría necesita el cambio de estado y el técnico. Nadie necesita la descripción larga de la avería, así que no viaja. Este criterio —enviar el mínimo necesario— reduce el ancho de banda y, sobre todo, disminuye el acoplamiento: cuanto menos comparta el evento, menos se romperán los consumidores cuando el emisor evolucione.

14.4 Paso 4.4 — SolicitudTicket.java, la entrada del formulario

```

1 package ec.edu.uteq.distapp.jms.dominio;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.NotNull;
5 import jakarta.validation.constraints.Size;
6
7 /**
8  * Lo que llega desde el formulario del navegador.
9  * No lleva id ni fecha: esos los genera el servidor, nunca el cliente.
10 */
11 public record SolicitudTicket(
12
13     @NotBlank(message = "El cliente es obligatorio")
14     @Size(max = 60)
15     String cliente,
16
17     @NotBlank(message = "El servicio es obligatorio")
18     String servicio,
19
20     @NotNull(message = "La prioridad es obligatoria")
21     Prioridad prioridad,
22
23     @NotBlank(message = "La descripcion es obligatoria")
24     @Size(max = 200)
25     String descripcion) {
26 }

```

Listing 14.4: dominio/SolicitudTicket.java

Nunca confíes en el identificador que envía el cliente

Si el navegador pudiera fijar el `id` del ticket, un usuario malicioso podría sobrescribir tickets ajenos enviando un identificador existente. Generar el identificador en el servidor es una regla básica de diseño seguro.

CAPÍTULO 15

La configuración JMS: el corazón del laboratorio

Este es el capítulo más importante del manual. Aquí es donde se decide, componente por componente, qué habla con la cola y qué habla con el tópico.

15.1 Paso 5.1 — JmsConfig.java

```
1 package ec.edu.uteq.distapp.jms.config;
2
3 import com.fasterxml.jackson.databind.ObjectMapper;
4 import jakarta.jms.ConnectionFactory;
5 import org.springframework.boot.autoconfigure.jms.
    DefaultJmsListenerContainerFactoryConfigurer;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.context.annotation.Configuration;
8 import org.springframework.context.annotation.Primary;
9 import org.springframework.jms.config.DefaultJmsListenerContainerFactory;
10 import org.springframework.jms.core.JmsTemplate;
11 import org.springframework.jms.support.converter.MappingJackson2MessageConverter;
12 import org.springframework.jms.support.converter.MessageConverter;
13 import org.springframework.jms.support.converter.MessageType;
14
15 @Configuration
16 public class JmsConfig {
17
18     // -----
19     // NOMBRES DE LOS DESTINOS
20     // Son constantes (static final) porque @JmsListener exige valores
21     // conocidos en tiempo de compilacion.
22     // -----
23
24     /** COLA: cada ticket lo atiende UN SOLO tecnico. */
25     public static final String COLA_TICKETS = "soporte.tickets.cola";
26
27     /** TOPICO: cada evento lo reciben TODOS los suscriptores. */
28     public static final String TOPICO_EVENTOS = "soporte.tickets.eventos";
29
30     // -----
31     // CONVERSOR: objeto Java <--> mensaje JMS
32     // -----
33
34     /**
35      * Convierte nuestros records a JSON dentro de un TextMessage.
36      *
37      * IMPORTANTE: recibimos por parametro el ObjectMapper que Spring Boot
38      * ya ha configurado. Si dejaramos que MappingJackson2MessageConverter
39      * creara el suyo propio, ese no tendria registrado el modulo de
40      * fechas de Java 8 y al enviar un Instant fallaria con:
41      * InvalidDefinitionException: Java 8 date/time type
42      * 'java.time.Instant' not supported by default
43      */
44     @Bean
45     public MessageConverter conversorJackson(ObjectMapper objectMapper) {
46         MappingJackson2MessageConverter conversor = new
47             MappingJackson2MessageConverter();
48         conversor.setObjectMapper(objectMapper);
49
50         // El cuerpo sera texto (JSON), no bytes ni objeto serializado.
51         conversor.setTargetType(MessageType.TEXT);
52
53         // El conversor escribe en esta propiedad el nombre de la clase,
54         // para que el consumidor sepa a que tipo deserializar.
55         conversor.setTypeIdPropertyName("_tipo");
56
57         return conversor;
58     }
59 }
```

```

59 // -----
60 // PLANTILLAS DE ENVIO
61 // Dos plantillas distintas porque pubSubDomain es una propiedad
62 // del objeto, no del metodo. Una plantilla habla SIEMPRE con colas
63 // o SIEMPRE con topicos.
64 // -----
65
66 /** Plantilla para COLAS. @Primary evita ambigüedad al inyectar. */
67 @Bean
68 @Primary
69 public JmsTemplate plantillaCola(ConnectionFactory fabricaConexiones,
70                                 MessageConverter conversorJackson) {
71     JmsTemplate plantilla = new JmsTemplate(fabricaConexiones);
72     plantilla.setPubSubDomain(false);           // <-- COLA
73     plantilla.setMessageConverter(conversorJackson);
74     return plantilla;
75 }
76
77 /** Plantilla para TOPICOS. */
78 @Bean
79 public JmsTemplate plantillaTopico(ConnectionFactory fabricaConexiones,
80                                   MessageConverter conversorJackson) {
81     JmsTemplate plantilla = new JmsTemplate(fabricaConexiones);
82     plantilla.setPubSubDomain(true);           // <-- TOPICO
83     plantilla.setMessageConverter(conversorJackson);
84     return plantilla;
85 }
86
87 // -----
88 // FABRICAS DE CONTENEDORES DE ESCUCHA
89 // Cada @JmsListener elige su fabrica por nombre. La fabrica decide
90 // si escucha una cola o un topico.
91 // -----
92
93 /** Fabrica para escuchar COLAS. */
94 @Bean
95 public DefaultJmsListenerContainerFactory fabricaCola(
96     ConnectionFactory fabricaConexiones,
97     DefaultJmsListenerContainerFactoryConfigurer configurador,
98     MessageConverter conversorJackson) {
99
100     DefaultJmsListenerContainerFactory fabrica = new
101         DefaultJmsListenerContainerFactory();
102
103     // Primero aplicamos lo que venga de application.properties...
104     configurador.configure(fabrica, fabricaConexiones);
105
106     // ...y DESPUES fijamos lo nuestro, para que no lo sobrescriba.
107     fabrica.setPubSubDomain(false);           // <-- COLA
108     fabrica.setMessageConverter(conversorJackson);
109
110     return fabrica;
111 }
112
113 /** Fabrica para escuchar TOPICOS con suscripcion NO durable. */
114 @Bean
115 public DefaultJmsListenerContainerFactory fabricaTopico(
116     ConnectionFactory fabricaConexiones,
117     DefaultJmsListenerContainerFactoryConfigurer configurador,
118     MessageConverter conversorJackson) {
119
120     DefaultJmsListenerContainerFactory fabrica = new
121         DefaultJmsListenerContainerFactory();
122     configurador.configure(fabrica, fabricaConexiones);
123
124     fabrica.setPubSubDomain(true);           // <-- TOPICO
125     fabrica.setSubscriptionDurable(false);   // no durable
126     fabrica.setMessageConverter(conversorJackson);
127
128     return fabrica;
129 }
130 /**

```

```
130  * Fabrica para escuchar TOPICOS con suscripcion DURABLE.
131  * El identificador de cliente debe ser unico en todo el broker.
132  */
133  @Bean
134  public DefaultJmsListenerContainerFactory fabricaTopicoDurable(
135      ConnectionFactory fabricaConexiones,
136      DefaultJmsListenerContainerFactoryConfigurer configurador,
137      MessageConverter conversorJackson) {
138
139      DefaultJmsListenerContainerFactory fabrica = new
140          DefaultJmsListenerContainerFactory();
141      configurador.configure(fabrica, fabricaConexiones);
142
143      fabrica.setPubSubDomain(true);           // <-- TOPICO
144      fabrica.setSubscriptionDurable(true);    // <-- DURABLE
145      fabrica.setClientId("auditoria-mesa-ayuda");
146      fabrica.setMessageConverter(conversorJackson);
147
148      return fabrica;
149  }
```

Listing 15.1: config/JmsConfig.java

15.2 Las cuatro decisiones de este archivo

15.2.1 Por qué dos JmsTemplate y no uno

`pubSubDomain` es un atributo del objeto `JmsTemplate`, no un parámetro de sus métodos. No existe ningún `convertAndSend` que reciba “y esto va a un tópico”. Por tanto, una plantilla habla siempre con colas o siempre con tópicos, y una aplicación que necesite los dos necesita dos plantillas.

Como habrá dos objetos del mismo tipo `JmsTemplate`, Spring no sabría cuál inyectar si alguien pidiera simplemente “un `JmsTemplate`”. Se resuelve de dos formas complementarias: `@Primary` designa la preferida por omisión, y `@Qualifier` permite pedir una concreta por su nombre, que es lo que haremos en el productor.

15.2.2 Por qué el orden importa en las fábricas

Fíjate en que `configurador.configure(...)` se llama *antes* de `setPubSubDomain`. Ese configurador aplica los valores que vengan de `application.properties`, incluida la propiedad global `spring.jms.pub-sub-domain`. Si invirtiéramos el orden, el configurador sobrescribiría nuestra decisión y volveríamos al problema de tener un solo dominio para toda la aplicación.

Este es un error real y silencioso

Invertir esas dos líneas no produce ningún error de compilación ni ninguna excepción. Simplemente, la mitad de la aplicación deja de recibir mensajes sin explicación aparente. Si algún día tu tópico no entrega nada, revisa este orden.

15.2.3 Por qué se inyecta el ObjectMapper

En una aplicación Spring Boot con `spring-boot-starter-web` conviven dos serializadores JSON potenciales: el `ObjectMapper` que Spring configura para los controladores REST, con soporte para fechas de Java 8 ya registrado, y el que `MappingJackson2MessageConverter` crearía por su cuenta si no le dijéramos nada, que no lo tiene. Como nuestros `record` llevan campos `Instant`, usar el segundo produce un fallo en el primer envío. Inyectar el primero resuelve el problema y además garantiza que el JSON de la API REST y el JSON de los mensajes tengan exactamente el mismo formato.

15.2.4 Por qué `setTypeIdPropertyName`

El conversor escribe en una propiedad del mensaje —aquí llamada `_tipo`— el nombre completo de la clase original. Al recibir, lee esa propiedad y sabe a qué tipo deserializar. Sin ella, el consumidor recibiría el JSON pero no sabría si construir un `Ticket` o un `EventoTicket`.

Si el productor y el consumidor están en aplicaciones distintas

Cuando emisor y receptor son procesos separados con paquetes distintos, el nombre completo de la clase no coincidirá y la deserialización fallará. La solución es registrar un mapa de tipos con `conversor.setTypeIdMappings(...)`, que traduce un identificador lógico corto como `"ticket"` a la clase local de cada lado. En nuestro laboratorio todo vive en un proyecto, así que no hace falta.

CAPÍTULO 16

El productor

16.1 Paso 6.1 — TicketProductor.java

```
1 package ec.edu.uteq.distapp.jms.mensajeria;
2
3 import ec.edu.uteq.distapp.jms.config.JmsConfig;
4 import ec.edu.uteq.distapp.jms.dominio.EventoTicket;
5 import ec.edu.uteq.distapp.jms.dominio.Ticket;
6 import org.slf4j.Logger;
7 import org.slf4j.LoggerFactory;
8 import org.springframework.beans.factory.annotation.Qualifier;
9 import org.springframework.jms.core.JmsTemplate;
10 import org.springframework.stereotype.Service;
11
12 /**
13  * Unico punto de la aplicacion que ENVIA mensajes.
14  * Concentrar aqui los dos envios deja a la vista la diferencia
15  * entre mandar un comando a una cola y publicar un evento en un topico.
16  */
17 @Service
18 public class TicketProductor {
19
20     private static final Logger log = LoggerFactory.getLogger(TicketProductor.class);
21
22     private final JmsTemplate plantillaCola;
23     private final JmsTemplate plantillaTopico;
24
25     /**
26      * @Qualifier es OBLIGATORIO aqui: hay dos beans de tipo JmsTemplate
27      * y sin el, Spring inyectaria la marcada con @Primary en los dos
28      * parametros, y los eventos acabarían en una cola.
29      */
30     public TicketProductor(@Qualifier("plantillaCola") JmsTemplate plantillaCola,
31                           @Qualifier("plantillaTopico") JmsTemplate plantillaTopico)
32     {
33         this.plantillaCola = plantillaCola;
34         this.plantillaTopico = plantillaTopico;
35     }
36
37     // -----
38     // ENVIO A LA COLA (punto a punto)
39     // -----
40
41     /**
42      * Deposita el ticket en la cola. Un solo tecnico lo recibira.
43      * El metodo retorna de inmediato: NO espera a que nadie lo atienda.
44      */
45     public void encolarTicket(Ticket ticket) {
46
47         plantillaCola.convertAndSend(
48             JmsConfig.COLA_TICKETS,           // destino
49             ticket,                           // objeto que se convertira a JSON
50             mensaje -> {
51                 // Duplicamos la prioridad como PROPIEDAD del mensaje
52                 // para que pueda usarse en selectores del lado del broker.
53                 mensaje.setStringProperty("prioridad", ticket.prioridad().name());
54                 mensaje.setStringProperty("servicio", ticket.servicio());
55                 return mensaje;
56             });
57
58         log.info("--> COLA | ticket {} encolado (prioridad {})",
59               ticket.id(), ticket.prioridad());
60     }
61
62     // -----
63     // PUBLICACION EN EL TOPICO (publicacion / suscripcion)
64     // -----
```

```
64
65  /**
66   * Publica el evento. Lo reciben TODOS los suscriptores activos.
67   * Si no hay ninguno, la operacion es valida y no ocurre nada.
68   */
69  public void publicarEvento(EventoTicket evento) {
70
71      plantillaTopico.convertAndSend(
72          JmsConfig.TOPICO_EVENTOS,
73          evento,
74          mensaje -> {
75              mensaje.setStringProperty("estadoNuevo", evento.estadoNuevo());
76              mensaje.setStringProperty("prioridad", evento.prioridad().name());
77              return mensaje;
78          });
79
80      log.info("--> TOPICO | evento {} -> {} del ticket {} publicado",
81          evento.estadoAnterior(), evento.estadoNuevo(), evento.ticketId());
82  }
83 }
```

Listing 16.1: mensajeria/TicketProductor.java

Sin @Qualifier, el laboratorio falla en silencio

Si olvidas los `@Qualifier`, Spring inyectará la plantilla marcada con `@Primary` en *ambos* campos. El código compilará, la aplicación arrancará y los eventos se enviarán a una *cola* llamada `soporte.tickets.eventos`. Los tres suscriptores, que escuchan el *tópico* del mismo nombre, no recibirán nada, y no habrá ninguna excepción que lo delate. Esta es la segunda causa más frecuente de “no me llega nada” en el laboratorio.

Qué hace exactamente `convertAndSend`

Es un método de conveniencia que encadena cuatro pasos: obtiene una sesión del *pool*, resuelve el destino usando `pubSubDomain` para decidir si es cola o tópico, aplica el `MessageConverter` para transformar el objeto en un `TextMessage` con JSON, ejecuta el *post-procesador* que le hemos pasado como lambda —donde añadimos las propiedades— y por último envía. Si algo falla, lanza una excepción no comprobada de la jerarquía `JmsException`.

CAPÍTULO 17

Los consumidores de la cola: los tres técnicos

17.1 Paso 7.1 — TecnicoConsumidor.java

```
1 package ec.edu.uteq.distapp.jms.mensajeria;
2
3 import ec.edu.uteq.distapp.jms.config.JmsConfig;
4 import ec.edu.uteq.distapp.jms.dominio.EventoTicket;
5 import ec.edu.uteq.distapp.jms.dominio.Ticket;
6 import ec.edu.uteq.distapp.jms.web.FilaTablero;
7 import ec.edu.uteq.distapp.jms.web.TableroSse;
8 import org.slf4j.Logger;
9 import org.slf4j.LoggerFactory;
10 import org.springframework.jms.annotation.JmsListener;
11 import org.springframework.stereotype.Component;
12
13 import java.time.Instant;
14 import java.util.concurrent.atomic.AtomicInteger;
15
16 /**
17  * Tres tecnicos escuchando LA MISMA COLA.
18  *
19  * Patron: Competing Consumers. El broker entrega cada ticket a UNO SOLO
20  * de los tres. Repartir la carga NO requiere ni una linea de logica
21  * nuestra: lo hace el broker.
22  */
23 @Component
24 public class TecnicoConsumidor {
25
26     private static final Logger log = LoggerFactory.getLogger(TecnicoConsumidor.class);
27     ;
28
29     private final TicketProductor productor;
30     private final TableroSse tablero;
31
32     // Contadores para demostrar el reparto. AtomicInteger es seguro
33     // entre hilos: cada tecnico corre en un hilo distinto.
34     private final AtomicInteger totalTec01 = new AtomicInteger();
35     private final AtomicInteger totalTec02 = new AtomicInteger();
36     private final AtomicInteger totalTec03 = new AtomicInteger();
37
38     public TecnicoConsumidor(TicketProductor productor, TableroSse tablero) {
39         this.productor = productor;
40         this.tablero = tablero;
41     }
42
43     // -----
44     // LOS TRES OYENTES: mismo destino, misma fabrica, metodos distintos
45     // -----
46
47     @JmsListener(destination = JmsConfig.COLA_TICKETS, containerFactory = "fabricaCola")
48     public void tecnicoUno(Ticket ticket) {
49         atender("TEC-01", ticket, totalTec01.incrementAndGet());
50     }
51
52     @JmsListener(destination = JmsConfig.COLA_TICKETS, containerFactory = "fabricaCola")
53     public void tecnicoDos(Ticket ticket) {
54         atender("TEC-02", ticket, totalTec02.incrementAndGet());
55     }
56
57     @JmsListener(destination = JmsConfig.COLA_TICKETS, containerFactory = "fabricaCola")
58     public void tecnicoTres(Ticket ticket) {
59         atender("TEC-03", ticket, totalTec03.incrementAndGet());
60     }
61 }
```

```

61 // -----
62 // LOGICA COMPARTIDA
63 // -----
64
65 private void atender(String tecnico, Ticket ticket, int acumulado) {
66
67     log.info("<-- COLA    | {} recibe el ticket {} (lleva {}) [hilo {}]",
68             tecnico, ticket.id(), acumulado, Thread.currentThread().getName());
69
70     // El navegador ve al instante quien lo tomo.
71     tablero.difundir("cola", new FilaTablero(
72         "cola", tecnico, ticket.id(), ticket.cliente(),
73         "Ticket asignado a " + tecnico, Instant.now().toString()));
74
75     // Simulamos el trabajo real del tecnico.
76     simularTrabajo(700);
77
78     // Terminado el trabajo, se PUBLICA el hecho en el topico.
79     // Aquí es donde la cola se convierte en topico.
80     productor.publicarEvento(new EventoTicket(
81         ticket.id(),
82         ticket.cliente(),
83         ticket.prioridad(),
84         "NUEVO",
85         "EN_ATENCION",
86         tecnico,
87         Instant.now()));
88 }
89
90 private void simularTrabajo(long milisegundos) {
91     try {
92         Thread.sleep(milisegundos);
93     } catch (InterruptedException e) {
94         // Buena practica: restaurar la marca de interrupcion del hilo.
95         Thread.currentThread().interrupt();
96     }
97 }
98 }

```

Listing 17.1: mensajeria/TecnicoConsumidor.java

17.2 Qué está pasando por debajo

17.2.1 @JmsListener no es una llamada, es una suscripción

Al arrancar la aplicación, Spring recorre todos los *beans* buscando métodos anotados con `@JmsListener`. Por cada uno crea un *contenedor de escucha*: un objeto que abre una conexión al *broker*, crea una sesión, crea un `MessageConsumer` sobre el destino indicado y arranca un hilo que se queda esperando. Cuando llega un mensaje, el contenedor lo convierte con el `MessageConverter` y llama a nuestro método.

Es decir, nuestro método nunca se invoca desde nuestro código: lo invoca la infraestructura, desde otro hilo. Por eso en el registro veremos nombres de hilo distintos para cada técnico.

17.2.2 Por qué los contadores son `AtomicInteger`

Los tres métodos corren en hilos distintos y de forma simultánea. Si los contadores fueran un simple `int` y dos hilos hicieran `contador++` a la vez, se perderían incrementos, porque `++` no es una operación atómica: son tres pasos (leer, sumar, escribir). `AtomicInteger` garantiza que `incrementAndGet()` sea indivisible.

17.2.3 Por qué el parámetro es directamente un Ticket

Podríamos haber declarado el parámetro como `jakarta.jms.Message` y deserializar a mano. Al declararlo como `Ticket`, Spring aplica automáticamente el `MessageConverter` configurado en la fábrica. Si necesitáramos además las propiedades o las cabeceras, la firma podría ser:

```
1 @JmsListener(destination = JmsConfig.COLA_TICKETS, containerFactory = "fabricaCola")
2 public void tecnicoUno(@Payload Ticket ticket,
3                        @Header("prioridad") String prioridad,
4                        @Header(JmsHeaders.REDELIVERED) boolean reintento) {
5     // ...
6 }
```

Listing 17.2: Firma alternativa con acceso a cabeceras

Alternativa: un solo método con concurrencia

En producción no se escriben tres métodos idénticos: se escribe uno y se le indica cuántas instancias concurrentes debe tener el contenedor.

```
@JmsListener(destination = JmsConfig.COLA_TICKETS,
              containerFactory = "fabricaCola",
              concurrency = "3-10")
public void atenderTicket(Ticket ticket) { ... }
```

La cadena "3-10" significa: mantén al menos tres consumidores y crece hasta diez si la cola se acumula. Aquí escribimos tres métodos separados solo para poder etiquetar a cada técnico con un nombre y hacer visible el reparto en pantalla.

CAPÍTULO 18

Los suscriptores del tópico

18.1 Paso 8.1 — SuscriptoresEventos.java

```
1 package ec.edu.uteq.distapp.jms.mensajeria;
2
3 import ec.edu.uteq.distapp.jms.config.JmsConfig;
4 import ec.edu.uteq.distapp.jms.dominio.EventoTicket;
5 import ec.edu.uteq.distapp.jms.web.FilaTablero;
6 import ec.edu.uteq.distapp.jms.web.TableroSse;
7 import org.slf4j.Logger;
8 import org.slf4j.LoggerFactory;
9 import org.springframework.jms.annotation.JmsListener;
10 import org.springframework.stereotype.Component;
11
12 import java.time.Instant;
13
14 /**
15  * Tres suscriptores INDEPENDIENTES al MISMO TOPICO.
16  *
17  * Cada uno recibe SU PROPIA COPIA de cada evento.
18  * Ninguno sabe que los otros existen. El publicador tampoco.
19  *
20  * Prueba de extensibilidad: para anadir un cuarto interesado basta con
21  * escribir otro metodo con @JmsListener aqui abajo. Ni el productor ni
22  * los otros tres suscriptores se enteran ni cambian.
23  */
24 @Component
25 public class SuscriptoresEventos {
26
27     private static final Logger log = LoggerFactory.getLogger(SuscriptoresEventos.
28         class);
29
30     private final TableroSse tablero;
31
32     public SuscriptoresEventos(TableroSse tablero) {
33         this.tablero = tablero;
34     }
35
36     // -----
37     // SUSCRIPTOR 1: notificacion al cliente. NO durable.
38     // Si estuvo caido, no queremos inundar al cliente con avisos viejos.
39     // -----
40     @JmsListener(destination = JmsConfig.TOPICO_EVENTOS, containerFactory = "
41         fabricaTopico")
42     public void notificarCliente(EventoTicket evento) {
43
44         String detalle = "Correo a " + evento.cliente()
45             + ": su ticket " + evento.ticketId() + " esta " + evento.estadoNuevo()
46             ;
47
48         log.info("<-- TOPICO | NOTIFICADOR | {}", detalle);
49         publicar("NOTIFICADOR", evento, detalle);
50     }
51
52     // -----
53     // SUSCRIPTOR 2: monitor de acuerdos de nivel de servicio. NO durable.
54     // -----
55     @JmsListener(destination = JmsConfig.TOPICO_EVENTOS, containerFactory = "
56         fabricaTopico")
57     public void monitorearSla(EventoTicket evento) {
58
59         int limite = evento.prioridad().minutosSla();
60         String detalle = "Cronometro SLA iniciado: " + limite
61             + " min para el ticket " + evento.ticketId();
62     }
```

```

61     log.info("<-- TOPICO | MONITOR SLA | {}\"", detalle);
62     publicar("MONITOR SLA", evento, detalle);
63 }
64
65 // -----
66 // SUSCRIPTOR 3: auditoria. DURABLE.
67 // Un registro de auditoria con huecos no sirve como evidencia,
68 // así que este suscriptor SI debe recibir lo que se publico
69 // mientras estuvo desconectado.
70 // -----
71
72 @JmsListener(destination = JmsConfig.TOPICO_EVENTOS,
73              containerFactory = "fabricaTopicoDurable",
74              subscription = "suscripcion-auditoria")
75 public void registrarAuditoria(EventoTicket evento) {
76
77     String detalle = "AUDIT " + evento.ocurridoEn() + " | ticket " + evento.
78                     ticketId()
79                     + " | " + evento.estadoAnterior() + " -> " + evento.estadoNuevo()
80                     + " | por " + evento.tecnico();
81
82     log.info("<-- TOPICO | AUDITORIA | {}\"", detalle);
83     publicar("AUDITORIA", evento, detalle);
84 }
85
86 // -----
87 private void publicar(String suscriptor, EventoTicket evento, String detalle) {
88     tablero.difundir("topico", new FilaTablero(
89         "topico", suscriptor, evento.ticketId(), evento.cliente(),
90         detalle, Instant.now().toString()));
91 }
92 }

```

Listing 18.1: mensajeria/SuscriptoresEventos.java

18.2 El detalle decisivo: subscription

El atributo `subscription` del tercer oyente le da nombre a la suscripción durable. Junto con el identificador de cliente que fijamos en `fabricaTopicoDurable`, forma la clave con la que el *broker* reconoce a este suscriptor cuando vuelve tras una desconexión.

Tres condiciones que deben cumplirse a la vez

Para que la suscripción durable funcione, las tres cosas siguientes deben ser ciertas simultáneamente: la fábrica debe tener `setSubscriptionDurable(true)`; la fábrica debe tener un `clientId` fijado y único; y el oyente debe declarar un `subscription` con nombre estable. Si falta cualquiera de las tres, Artemis creará una suscripción no durable sin quejarse, y el Experimento 22.5 fallará sin dar ninguna pista.

Cómo comprobarlo en la consola

Entra en la consola de Artemis y navega hasta la dirección `soporte.tickets.eventos`. Verás tres colas hijas. Dos tendrán nombres largos generados automáticamente: son los suscriptores no durables. Una tendrá un nombre que contiene `auditoria-mesa-ayuda` y `suscripcion-auditoria`: esa es la durable. Si detienes la aplicación, las dos primeras desaparecen y la tercera permanece. Ese es exactamente el concepto, hecho visible.

CAPÍTULO 19

La capa web: REST y eventos hacia el navegador

Necesitamos que el navegador *vea* lo que ocurre en el instante en que ocurre. La técnica que usaremos se llama SSE (*Server-Sent Events*): un canal HTTP que el servidor deja abierto y por el que va empujando eventos.

Por qué SSE y no WebSocket ni sondeo

El sondeo periódico —preguntar cada segundo si hay novedades— desperdicia peticiones y añade latencia. WebSocket es bidireccional y potente, pero requiere una configuración adicional y un protocolo encima, como STOMP. SSE es unidireccional del servidor al cliente, que es exactamente lo que necesitamos, viaja sobre HTTP normal, se implementa en el navegador con una sola clase (`EventSource`) y reconecta solo si se corta. Para un tablero de observación es la herramienta correcta.

19.1 Paso 9.1 — `FilaTablero.java`

```
1 package ec.edu.uteq.distapp.jms.web;
2
3 /**
4  * Una linea del tablero del navegador.
5  * canal vale "cola" o "topico" y decide en que columnas se pinta.
6  */
7 public record FilaTablero(
8     String canal,
9     String actor,
10    String ticketId,
11    String cliente,
12    String detalle,
13    String instante) {
14 }
```

Listing 19.1: web/FilaTablero.java

19.2 Paso 9.2 — `TableroSse.java`

```
1 package ec.edu.uteq.distapp.jms.web;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.stereotype.Component;
6 import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;
7
8 import java.io.IOException;
9 import java.util.List;
10 import java.util.concurrent.CopyOnWriteArrayList;
11
12 /**
13  * Mantiene la lista de navegadores conectados y les empuja eventos.
14  *
15  * No tiene NADA que ver con JMS: es solo el cable entre el backend y
16  * la pantalla. Se ha separado a proposito para que quede claro donde
17  * termina la mensajería y donde empieza la interfaz.
18  */
19 @Component
20 public class TableroSse {
21
22     private static final Logger log = LoggerFactory.getLogger(TableroSse.class);
23
24     /**
```

```

25      * CopyOnWriteArrayList es una lista segura entre hilos, optimizada
26      * para muchas lecturas y pocas escrituras: justo nuestro caso,
27      * porque los seis consumidores leen constantemente y solo se
28      * escribe cuando un navegador se conecta o se va.
29      */
30      private final List<SseEmitter> emisores = new CopyOnWriteArrayList<>();
31
32      /** Registra un navegador nuevo y le devuelve su canal. */
33      public SseEmitter registrar() {
34
35          // OL significa "sin tiempo de expiracion".
36          SseEmitter emisor = new SseEmitter(OL);
37
38          emisor.onCompletion(() -> emisores.remove(emisor));
39          emisor.onTimeout(() -> emisores.remove(emisor));
40          emisor.onError(error -> emisores.remove(emisor));
41
42          emisores.add(emisor);
43          log.info("Tablero: navegador conectado. Total: {}", emisores.size());
44          return emisor;
45      }
46
47      /**
48       * Envía una fila a TODOS los navegadores conectados.
49       * El parametro canal se convierte en el nombre del evento SSE,
50       * que el JavaScript usará para decidir en qué columna pintarlo.
51       */
52      public void difundir(String canal, FilaTablero fila) {
53          for (SseEmitter emisor : emisores) {
54              try {
55                  emisor.send(SseEmitter.event().name(canal).data(fila));
56              } catch (IOException e) {
57                  // El navegador cerró la pestaña. No es un error real.
58                  emisores.remove(emisor);
59              }
60          }
61      }
62  }

```

Listing 19.2: web/TableroSse.java

Una analogía que ayuda

Fíjate en que `TableroSse` es, en pequeño, un tópicos: mantiene una lista de suscriptores y entrega una copia a cada uno. La diferencia es que vive dentro de un solo proceso, en memoria, y no sobrevive a un reinicio. Eso es justamente lo que un *broker* añade: persistencia, distribución entre máquinas y garantías de entrega. Comparar ambas piezas es una buena manera de entender qué se está comprando al introducir un intermediario.

19.3 Paso 9.3 — TicketControlador.java

```

1  package ec.edu.uteq.distapp.jms.web;
2
3  import ec.edu.uteq.distapp.jms.dominio.SolicitudTicket;
4  import ec.edu.uteq.distapp.jms.dominio.Ticket;
5  import ec.edu.uteq.distapp.jms.mensajeria.TicketProductor;
6  import jakarta.validation.Valid;
7  import org.springframework.http.MediaType;
8  import org.springframework.http.ResponseEntity;
9  import org.springframework.web.bind.annotation.*;
10 import org.springframework.web.servlet.mvc.method.annotation.SseEmitter;
11
12 import java.time.Instant;
13 import java.util.concurrent.atomic.AtomicInteger;
14
15 @RestController
16 @RequestMapping("/api")
17 public class TicketControlador {

```

```

18
19 private final TicketProductor productor;
20 private final TableroSse tablero;
21
22 // Numerador simple de tickets. En produccion vendria de la base de datos.
23 private final AtomicInteger secuencia = new AtomicInteger(1000);
24
25 public TicketControlador(TicketProductor productor, TableroSse tablero) {
26     this.productor = productor;
27     this.tablero = tablero;
28 }
29
30 /**
31  * Recibe el formulario y ENCOLA el ticket.
32  *
33  * Observa que el metodo responde de inmediato, sin esperar a que
34  * ningun tecnico lo atienda: eso es el desacoplamiento de
35  * sincronizacion en accion.
36  */
37 @PostMapping(path = "/tickets", consumes = MediaType.APPLICATION_JSON_VALUE)
38 public ResponseEntity<Ticket> crear(@Valid @RequestBody SolicitudTicket solicitud)
39 {
40     Ticket ticket = new Ticket(
41         "TK-" + secuencia.incrementAndGet(),
42         solicitud.cliente(),
43         solicitud.servicio(),
44         solicitud.prioridad(),
45         solicitud.descripcion(),
46         Instant.now());
47
48     productor.encolarTicket(ticket);
49
50     // 202 Accepted es el codigo correcto: "lo he aceptado y lo
51     // procesare", no "ya esta hecho". Devolver 200 OK seria mentir.
52     return ResponseEntity.accepted().body(ticket);
53 }
54
55 /**
56  * Genera varios tickets de golpe para ver el reparto entre tecnicos.
57  */
58 @PostMapping("/tickets/lote/{cantidad}")
59 public ResponseEntity<String> lote(@PathVariable int cantidad) {
60
61     int total = Math.min(Math.max(cantidad, 1), 20); // limite de seguridad
62
63     for (int i = 1; i <= total; i++) {
64         productor.encolarTicket(new Ticket(
65             "TK-" + secuencia.incrementAndGet(),
66             "Cliente " + i,
67             "Internet Fibra",
68             ec.edu.uteq.distapp.jms.dominio.Prioridad.MEDIA,
69             "Averia generada automaticamente numero " + i,
70             Instant.now()));
71     }
72     return ResponseEntity.accepted().body(total + " tickets encolados");
73 }
74
75 /**
76  * Canal de eventos hacia el navegador.
77  * El tipo de contenido text/event-stream es lo que convierte esta
78  * respuesta en un canal SSE.
79  */
80 @GetMapping(path = "/flujo", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
81 public SseEmitter flujo() {
82     return tablero.registrar();
83 }
84 }

```

Listing 19.3: web/TicketControlador.java

Por qué 202 y no 200

El código HTTP 202 (*Accepted*) significa literalmente “he recibido tu petición y la procesaré, pero todavía no está hecha”. Es el código semánticamente correcto para toda API que delega el trabajo a una cola. Devolver 200 OK daría a entender al cliente que el ticket ya fue atendido, lo cual es falso: en ese instante solo está encolado. Esta clase de precisión importa cuando otro equipo consume tu API.

19.4 Paso 9.4 — La clase de arranque

IntelliJ ya la generó. Compruébala:

```

1 package ec.edu.uteq.distapp.jms;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 /**
7  * Punto de entrada.
8  *
9  * @SpringBootApplication equivale a tres anotaciones a la vez:
10 *   @Configuration      -> esta clase puede definir beans
11 *   @EnableAutoConfiguration -> configura Tomcat, JMS, Jackson... solo
12 *   @ComponentScan      -> busca @Component, @Service, @RestController
13 *                       en ESTE paquete y en todos los de debajo
14 *
15 * De ahí que TODO el código deba colgar de ec.edu.uteq.distapp.jms:
16 * una clase colocada fuera de ese árbol sería ignorada por completo.
17 */
18 @SpringBootApplication
19 public class MesaAyudaJmsApplication {
20
21     public static void main(String[] args) {
22         SpringApplication.run(MesaAyudaJmsApplication.class, args);
23     }
24 }

```

Listing 19.4: MesaAyudaJmsApplication.java

No hace falta @EnableJms

En Spring Boot, la presencia de `spring-boot-starter-artemis` activa automáticamente el procesamiento de `@JmsListener`. Añadir `@EnableJms` a mano no rompe nada, pero es redundante. Lo que sí rompe todo es colocar una clase con `@JmsListener` fuera del paquete raíz: no se detectará y el oyente jamás se suscribirá, sin mensaje de error alguno.

CAPÍTULO 20

El frontend

Todo lo que pongamos en `src/main/resources/static` lo sirve Spring Boot automáticamente en la raíz del sitio. No hace falta Node, ni npm, ni ningún empaquetador.

20.1 Paso 10.1 — `index.html`

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Mesa de ayuda ISP - JMS: cola y topico</title>
7   <link rel="stylesheet" href="estilos.css">
8 </head>
9 <body>
10
11 <header>
12   <h1>Mesa de ayuda ISP</h1>
13   <p class="sub">Demostracion JMS: <strong>cola</strong> (uno) frente a
14     <strong>topico</strong> (todos)</p>
15   <span id="estado" class="estado desconectado">Conectando...</span>
16 </header>
17
18 <main>
19
20   <!-- ===== COLUMNA 1: emisor ===== -->
21   <section class="panel">
22     <h2>1. Registrar averia</h2>
23
24     <label for="cliente">Cliente</label>
25     <input id="cliente" type="text" value="Maria Zambrano">
26
27     <label for="servicio">Servicio</label>
28     <select id="servicio">
29       <option>Internet Fibra</option>
30       <option>Internet Radioenlace</option>
31       <option>Television IP</option>
32       <option>Telefonia</option>
33     </select>
34
35     <label for="prioridad">Prioridad</label>
36     <select id="prioridad">
37       <option value="BAJA">BAJA (8 h)</option>
38       <option value="MEDIA" selected>MEDIA (4 h)</option>
39       <option value="ALTA">ALTA (1 h)</option>
40       <option value="CRITICA">CRITICA (15 min)</option>
41     </select>
42
43     <label for="descripcion">Descripcion</label>
44     <textarea id="descripcion" rows="3">Sin servicio desde las 14:00</textarea>
45
46     <button id="btnEnviar" class="primario">Enviar a la COLA</button>
47     <button id="btnLote" class="secundario">Enviar 9 tickets de golpe</button>
48     <button id="btnLimpiar" class="terciario">Limpiar tableros</button>
49
50     <p class="ayuda">
51       El boton de lote sirve para ver el reparto: nueve tickets
52       deberian repartirse aproximadamente a partes iguales entre
53       los tres tecnicos.
54     </p>
55   </section>
56
57   <!-- ===== COLUMNA 2: la cola ===== -->
58   <section class="panel cola">
59     <h2>2. COLA <span class="etiqueta">punto a punto</span></h2>
60     <p class="regla">Cada ticket lo recibe <strong>UN SOLO</strong> tecnico.</p>
```

```

61
62     <div class="contadores">
63         <div class="contador"><span id="cTec01">0</span><small>TEC-01</small></div>
64         <div class="contador"><span id="cTec02">0</span><small>TEC-02</small></div>
65         <div class="contador"><span id="cTec03">0</span><small>TEC-03</small></div>
66     </div>
67
68     <ul id="listaCola" class="registro"></ul>
69 </section>
70
71 <!-- ===== COLUMNA 3: el topico ===== -->
72 <section class="panel topico">
73     <h2>3. TOPICO <span class="etiqueta">publicacion / suscripcion</span></h2>
74     <p class="regla">Cada evento lo reciben <strong>LOS TRES</strong> suscriptores
75     .</p>
76
77     <div class="contadores">
78         <div class="contador"><span id="cNot">0</span><small>NOTIFICADOR</small></div>
79         <div class="contador"><span id="cSla">0</span><small>MONITOR SLA</small></div>
80         <div class="contador"><span id="cAud">0</span><small>AUDITORIA</small></div>
81     </div>
82
83     <ul id="listaTopico" class="registro"></ul>
84 </section>
85 </main>
86
87 <footer>
88     <p>Aplicaciones Distribuidas (ISR-701) - Facultad de Ciencias de la
89         Computacion - UTEQ</p>
90 </footer>
91
92 <script src="app.js"></script>
93 </body>
94 </html>

```

Listing 20.1: static/index.html

20.2 Paso 10.2 — estilos.css

```

1  :root {
2      --azul: #0E3F7E;
3      --medio: #1E5AA8;
4      --verde: #4FAE60;
5      --ambar: #F2A413;
6      --gris: #F4F6F9;
7      --rojo: #B3261E;
8      --borde: #D8DEE7;
9  }
10
11  * { box-sizing: border-box; }
12
13  body {
14      margin: 0;
15      font-family: "Segoe UI", system-ui, sans-serif;
16      background: var(--gris);
17      color: #1B2430;
18  }
19
20  header {
21      background: var(--azul);
22      color: #fff;
23      padding: 18px 26px;
24      position: relative;
25  }

```

```

26 header h1 { margin: 0; font-size: 1.5rem; }
27 header .sub { margin: 4px 0 0; opacity: .85; font-size: .9rem; }
28
29 .estado {
30     position: absolute; right: 26px; top: 22px;
31     padding: 5px 12px; border-radius: 12px;
32     font-size: .78rem; font-weight: 600;
33 }
34 .estado.conectado { background: var(--verde); color: #fff; }
35 .estado.desconectado { background: var(--rojo); color: #fff; }
36
37 main {
38     display: grid;
39     grid-template-columns: 1fr 1.25fr 1.25fr;
40     gap: 16px;
41     padding: 18px 26px;
42     align-items: start;
43 }
44
45 .panel {
46     background: #fff;
47     border: 1px solid var(--borde);
48     border-radius: 8px;
49     padding: 16px;
50 }
51 .panel h2 { margin: 0 0 4px; font-size: 1.05rem; color: var(--azul); }
52 .panel.cola { border-top: 4px solid var(--medio); }
53 .panel.topico { border-top: 4px solid var(--ambar); }
54
55 .etiqueta {
56     font-size: .68rem; font-weight: 600; text-transform: uppercase;
57     background: var(--gris); border: 1px solid var(--borde);
58     padding: 2px 7px; border-radius: 10px; vertical-align: middle;
59 }
60 .regla { font-size: .84rem; color: #55617A; margin: 0 0 12px; }
61 .ayuda { font-size: .78rem; color: #6B7688; margin-top: 12px; }
62
63 label {
64     display: block; margin: 10px 0 4px;
65     font-size: .78rem; font-weight: 600; color: #55617A;
66 }
67 input, select, textarea {
68     width: 100%; padding: 8px 10px;
69     border: 1px solid var(--borde); border-radius: 5px;
70     font-family: inherit; font-size: .88rem;
71 }
72
73 button {
74     width: 100%; margin-top: 10px; padding: 10px;
75     border: 0; border-radius: 5px; cursor: pointer;
76     font-size: .88rem; font-weight: 600;
77 }
78 .primario { background: var(--azul); color: #fff; }
79 .secundario { background: var(--ambar); color: #3A2A00; }
80 .terciario { background: #fff; color: #55617A; border: 1px solid var(--borde); }
81 button:hover { opacity: .9; }
82
83 .contadores { display: flex; gap: 8px; margin-bottom: 12px; }
84 .contador {
85     flex: 1; text-align: center; padding: 8px 4px;
86     background: var(--gris); border: 1px solid var(--borde); border-radius: 6px;
87 }
88 .contador span { display: block; font-size: 1.4rem; font-weight: 700; color: var(--azul); }
89 .contador small { font-size: .64rem; color: #6B7688; letter-spacing: .5px; }
90
91 .registro {
92     list-style: none; margin: 0; padding: 0;
93     max-height: 420px; overflow-y: auto;
94     font-size: .8rem;
95 }
96 .registro li {
97     padding: 7px 9px; margin-bottom: 5px; border-radius: 5px;

```

```

98     background: var(--gris); border-left: 3px solid var(--medio);
99     animation: entra .28s ease-out;
100 }
101 .panel.topico .registro li { border-left-color: var(--ambar); }
102 .registro .actor { font-weight: 700; color: var(--azul); }
103 .registro .hora { float: right; color: #8A93A3; font-size: .72rem; }
104
105 @keyframes entra {
106     from { opacity: 0; transform: translateY(-6px); }
107     to   { opacity: 1; transform: translateY(0); }
108 }
109
110 footer {
111     text-align: center; padding: 14px;
112     font-size: .76rem; color: #6B7688;
113 }
114
115 @media (max-width: 980px) {
116     main { grid-template-columns: 1fr; }
117 }

```

Listing 20.2: static/estilos.css

20.3 Paso 10.3 — app.js

```

1  // =====
2  // 1. CANAL DE EVENTOS DEL SERVIDOR (SSE)
3  // =====
4
5  // EventSource abre una conexión HTTP que el servidor NO cierra, y por
6  // la que va empujando eventos. Reconecta sola si se corta.
7  const flujo = new EventSource("/api/flujo");
8
9  const estado = document.getElementById("estado");
10
11  flujo.onopen = () => {
12      estado.textContent = "Conectado";
13      estado.className = "estado conectado";
14  };
15
16  flujo.onerror = () => {
17      estado.textContent = "Desconectado";
18      estado.className = "estado desconectado";
19  };
20
21  // El nombre del evento ("cola" u "topico") lo puso el backend en
22  // SseEmitter.event().name(...). Por eso podemos escuchar cada uno
23  // por separado y pintarlos en columnas distintas.
24  flujo.addEventListener("cola", e => pintar("cola", JSON.parse(e.data)));
25  flujo.addEventListener("topico", e => pintar("topico", JSON.parse(e.data)));
26
27  // =====
28  // 2. PINTADO DE FILAS
29  // =====
30
31  const contadores = {
32      "TEC-01": document.getElementById("cTec01"),
33      "TEC-02": document.getElementById("cTec02"),
34      "TEC-03": document.getElementById("cTec03"),
35      "NOTIFICADOR": document.getElementById("cNot"),
36      "MONITOR SLA": document.getElementById("cSla"),
37      "AUDITORIA": document.getElementById("cAud")
38  };
39
40  function pintar(canal, fila) {
41
42      const lista = document.getElementById(
43          canal === "cola" ? "listaCola" : "listaTopico");
44
45      const li = document.createElement("li");
46      li.innerHTML =

```



```

47     '<span class="hora">' + hora(fila.instante) + '</span>' +
48     '<span class="actor">' + fila.actor + '</span> &middot; ' +
49     fila.ticketId + '<br>' + fila.detalle;
50
51     // insertBefore con firstChild pone lo nuevo ARRIBA.
52     lista.insertBefore(li, lista.firstChild);
53
54     const c = contadores[fila.actor];
55     if (c) c.textContent = parseInt(c.textContent, 10) + 1;
56 }
57
58 function hora(instante) {
59     // El backend envia un Instant serializado en ISO-8601.
60     return new Date(instante).toLocaleTimeString("es-EC");
61 }
62
63 // =====
64 // 3. ENVIO DE TICKETS
65 // =====
66
67 document.getElementById("btnEnviar").addEventListener("click", async () => {
68
69     const cuerpo = {
70         cliente: document.getElementById("cliente").value,
71         servicio: document.getElementById("servicio").value,
72         prioridad: document.getElementById("prioridad").value,
73         descripcion: document.getElementById("descripcion").value
74     };
75
76     const respuesta = await fetch("/api/tickets", {
77         method: "POST",
78         headers: { "Content-Type": "application/json" },
79         body: JSON.stringify(cuerpo)
80     });
81
82     if (!respuesta.ok) {
83         alert("Error al enviar el ticket: " + respuesta.status);
84     }
85     // No hace falta hacer nada mas: la respuesta visible llegara por SSE
86     // cuando un tecnico tome el ticket. Eso ES el desacoplamiento.
87 });
88
89 document.getElementById("btnLote").addEventListener("click", async () => {
90     await fetch("/api/tickets/lote/9", { method: "POST" });
91 });
92
93 document.getElementById("btnLimpiar").addEventListener("click", () => {
94     document.getElementById("listaCola").innerHTML = "";
95     document.getElementById("listaTopico").innerHTML = "";
96     Object.values(contadores).forEach(c => c.textContent = "0");
97 });

```

Listing 20.3: static/app.js

Lo que el frontend enseña sin decirlo

Cuando pulses “Enviar a la COLA”, la respuesta HTTP volverá en pocos milisegundos, pero la primera línea del tablero de la cola aparecerá un instante después, y la del tópico casi un segundo más tarde. Ese desfase visible es el desacoplamiento de sincronización: el navegador no esperó a que el técnico terminara. Merece la pena hacer notar esto en clase, porque es la primera vez que muchos estudiantes *ven* que una petición puede terminar antes que el trabajo que desencadena.

Parte V

Ejecución, experimentos y diagnóstico

CAPÍTULO 21

Ejecutar la aplicación

21.1 Paso 11.1 — Comprobar que el broker está vivo

```
1 C:\labs\distapp-jms> docker compose ps
```

Listing 21.1: Verificación previa

El contenedor `artemis-distapp` debe aparecer en estado `running`. Si no, arráncalo con `docker compose up -d`.

21.2 Paso 11.2 — Arrancar desde IntelliJ

Abre `MesaAyudaJmsApplication.java`, haz clic en el triángulo verde del margen izquierdo, junto a la declaración de la clase o del método `main`, y elige `Run`.

También puedes usar Maven desde la terminal:

```
1 C:\labs\distapp-jms\mesa-ayuda-jms> mvnw.cmd spring-boot:run
```

Listing 21.2: Arranque desde la línea de comandos

21.3 Paso 11.3 — Leer el arranque

En la consola deben aparecer, entre otras, líneas equivalentes a estas:

```
1 Tomcat initialized with port 8080 (http)
2 Started MesaAyudaJmsApplication in 2.841 seconds
```

Listing 21.3: Salida esperada al arrancar

Lo que **no** debe aparecer es ninguna traza que mencione `Connection refused`, `AMQ219007` o `Could not refresh JMS Connection`: todas ellas significan que la aplicación no encuentra el *broker*.

21.4 Paso 11.4 — Abrir el tablero

Ve a `http://localhost:8080`. Deberías ver las tres columnas y, arriba a la derecha, la etiqueta verde `Conectado`. Si dice `Desconectado` en rojo, el canal SSE no se estableció; revisa la consola del navegador con `F12`.

CAPÍTULO 22

Siete experimentos

Los experimentos están ordenados: cada uno se apoya en lo aprendido en el anterior. Conviene hacerlos con la consola de IntelliJ y el navegador visibles a la vez.

22.1 Experimento 1 — La cola reparte

Acción. Pulsa “Enviar 9 tickets de golpe”.

Qué observar. En la columna de la cola, los tres contadores suben hasta sumar nueve, repartidos aproximadamente a partes iguales. Ningún ticket aparece dos veces. En la consola, cada línea `<- COLA` lleva un nombre de hilo distinto.

Qué demuestra. Entrega única y balanceo automático. No hemos escrito ni una línea de lógica de reparto: la hace el *broker*.

Si el reparto sale muy desigual

Es normal que no sea exactamente 3-3-3. El *broker* entrega al primer consumidor disponible, y como cada técnico tarda 700 milisegundos simulados, el reparto depende del azar de la planificación de hilos. Lo relevante no es la igualdad exacta, sino que la *suma* sea exactamente nueve.

22.2 Experimento 2 — El tópico difunde

Acción. Envía un solo ticket con el botón principal.

Qué observar. En la columna de la cola aparece *una* línea. Al cabo de menos de un segundo, en la columna del tópico aparecen *tres* líneas —notificador, monitor de SLA y auditoría— todas referidas al *mismo* ticket. Los tres contadores del tópico suben en uno.

Qué demuestra. Un único evento publicado se convierte en tres entregas independientes. Es la diferencia semántica esencial frente a la cola.

22.3 Experimento 3 — Extensibilidad sin tocar al emisor

Acción. Añade un cuarto suscriptor al final de `SuscriptoresEventos`:

```
1 @JmsListener(destination = JmsConfig.TOPICO_EVENTOS, containerFactory = "fabricaTopico")
2 public void alimentarEstadisticas(EventoTicket evento) {
3     String detalle = "Métrica registrada para " + evento.tecnico();
4     log.info("<-- TOPICO | ESTADISTICAS | {}", detalle);
5     publicar("ESTADISTICAS", evento, detalle);
6 }
```

Listing 22.1: Cuarto suscriptor: tablero de estadísticas

Guarda, deja que DevTools reinicie y envía otro ticket.

Qué observar. El nuevo suscriptor recibe los eventos inmediatamente. Ahora cuenta cuántos archivos has tenido que modificar: exactamente uno, y no es el productor. `TicketProductor`, `TecnicoConsumidor` y `JmsConfig` siguen intactos.

Qué demuestra. Desacoplamiento espacial. Con la solución síncrona del Listado 1.1 habría hecho falta modificar el emisor y volver a desplegarlo.

22.4 Experimento 4 — La cola conserva; el tópico no

Acción. Detén la aplicación en IntelliJ, pero *deja el broker corriendo*. Entra en la consola de Artemis, busca la dirección `soporte.tickets.colas` y usa la opción de envío manual para depositar tres mensajes. Después arranca de nuevo la aplicación.

Qué observar. En cuanto la aplicación arranca, los tres mensajes se consumen de golpe. Estaban esperando.

Ahora repite el ejercicio pero enviando a la dirección `soporte.tickets.eventos`, con enrutamiento MULTICAST. Al arrancar la aplicación, el notificador y el monitor de SLA *no* reciben nada: sus suscripciones no durables no existían en el momento de la publicación.

Qué demuestra. La cola almacena hasta que alguien consume; el tópico no durable descarta si no hay nadie escuchando. Este es el experimento que más veces hay que repetir para que el concepto se asiente.

22.5 Experimento 5 — La suscripción durable sí conserva

Acción. En el experimento anterior, fíjate en el suscriptor de auditoría.

Qué observar. A diferencia de los otros dos, la auditoría *sí* recibe los eventos publicados mientras la aplicación estuvo apagada. En la consola de Artemis, la cola interna correspondiente a `auditoria-mesa-ayuda` sigue existiendo con la aplicación parada, y su contador de mensajes pendientes crece.

Qué demuestra. Que la durabilidad de una suscripción es una decisión por suscriptor, no una propiedad del tópico. Tres suscriptores del mismo tópico pueden tener garantías distintas, y eso es exactamente lo que el negocio pedía: al cliente no le interesan los avisos atrasados, pero a la auditoría sí.

22.6 Experimento 6 — Selectores

Acción. Añade un suscriptor que solo se interese por lo crítico:

```
1 @JmsListener(destination = JmsConfig.TOPICO_EVENTOS,
2               containerFactory = "fabricaTopico",
3               selector = "prioridad = 'CRITICA'")
4 public void escalarCriticos(EventoTicket evento) {
5     log.info("<-- TOPICO | ESCALADO | Aviso al supervisor: {}", evento.ticketId()
6             );
7     publicar("ESCALADO", evento, "Escalado al supervisor de turno");
8 }
```

Listing 22.2: Suscriptor con selector

Envía dos tickets, uno con prioridad `MEDIA` y otro con `CRITICA`.

Qué observar. El nuevo suscriptor solo reacciona al segundo. Y el filtrado ocurrió *en el broker*: el mensaje de prioridad media nunca viajó por la red hasta este consumidor.

Qué demuestra. Que se puede filtrar sin ensuciar el código con condicionales, y que la propiedad que añadimos en el productor no era decorativa. Comprueba además que si intentas escribir `selector = "descripcion LIKE '%fibra%'"` no funcionará: la descripción está en el cuerpo, y los selectores no ven el cuerpo.

22.7 Experimento 7 — Reintento y cola de mensajes muertos

Acción. Provoca un fallo controlado en el técnico uno:

```
1 @JmsListener(destination = JmsConfig.COLA_TICKETS, containerFactory = "fabricaCola")
```



```

2 public void tecnicoUno(Ticket ticket) {
3     if (ticket.prioridad() == Prioridad.CRITICA) {
4         throw new IllegalStateException("Fallo simulado atendiendo " + ticket.id());
5     }
6     atender("TEC-01", ticket, totalTec01.incrementAndGet());
7 }

```

Listing 22.3: Fallo deliberado para observar el reintento

Envía un ticket con prioridad CRITICA.

Qué observar. En la consola aparece la excepción, y acto seguido el mismo ticket vuelve a entregarse. Puede caer en el mismo técnico o en otro. Tras varios intentos, Artemis lo mueve a la cola DLQ, que puedes ver en la consola de administración.

Qué demuestra. Que con AUTO_ACKNOWLEDGE lanzar una excepción equivale a rechazar el mensaje, que el trabajo no se pierde por un fallo transitorio, y que existe un límite: los mensajes irrecuperables acaban en la DLQ en lugar de bloquear la cola para siempre.

Detectar una reentrega desde el código

Añade el parámetro de cabecera `@Header(JmsHeaders.REDELIVERED)` `boolean reintento` a la firma del método. Si vale `true`, ese mensaje ya falló antes. Es la manera estándar de decidir si conviene volver a intentarlo o desviarlo directamente a un tratamiento especial.

Exp.	Qué se hace	Concepto que queda demostrado
1	Nueve tickets a la vez	Entrega única y balanceo automático en la cola
2	Un ticket, tres reacciones	Difusión uno a muchos en el tópico
3	Añadir un suscriptor	Desacoplamiento espacial: el emisor no cambia
4	Publicar con la app parada	La cola conserva; el tópico no durable descarta
5	Observar la auditoría	La durabilidad se decide por suscriptor
6	Filtrar por prioridad	Selectores del lado del servidor sobre propiedades
7	Lanzar una excepción	Reintento, idempotencia y cola de mensajes muertos

CAPÍTULO 23

Errores frecuentes y cómo diagnosticarlos

23.1 Método general de diagnóstico

Ante cualquier fallo, recorre estas cuatro preguntas en orden. Casi siempre el problema se localiza antes de llegar a la cuarta.

1. ¿Está el *broker* arriba? `docker compose ps`.
2. ¿Llegó el mensaje al *broker*? Míralo en la consola de Artemis: si el contador de mensajes de la dirección no sube, el fallo está en el productor.
3. ¿Hay consumidores conectados a esa dirección? La consola lo indica. Si el mensaje llegó pero nadie lo consume, el fallo está en el oyente o en el dominio.
4. ¿Coinciden productor y consumidor en el dominio? Este es el fallo silencioso por excelencia.

23.2 Catálogo de errores

Síntoma o mensaje	Causa real	Solución
Could not refresh JMS Connection, Connection refused	El <i>broker</i> no está arrancado o el puerto no coincide.	<code>docker compose up -d</code> ; comprobar que <code>broker-url</code> apunta a <code>tcp://localhost:61616</code> .
AMQ229031: Unable to validate user	Usuario o contraseña distintos de los del <code>docker-compose.yml</code> .	Igualar <code>spring.artemis.user</code> y <code>password</code> con <code>ARTEMIS_USER</code> y <code>ARTEMIS_PASSWORD</code> .
package javax.jms does not exist	Código copiado de un tutorial de Spring Boot 2.x.	Cambiar todos los <code>import javax.jms</code> por <code>jakarta.jms</code> .
Java 8 date/time type java.time.Instant not supported by default	El conversor creó su propio <code>ObjectMapper</code> sin el módulo de fechas.	Injectar el <code>ObjectMapper</code> de Spring con <code>setObjectMapper</code> , como en el Listado 15.1.
El productor envía, no hay excepción, pero nadie recibe .	Desajuste de dominio: uno usa cola y el otro tópico.	Comprobar <code>pubSubDomain</code> en la plantilla y en la fábrica; comprobar los <code>@Qualifier</code> .
Los tres suscriptores del tópico reciben, pero los técnicos reciben el mismo ticket tres veces .	La cola se configuró como tópico.	<code>fabricaCola</code> debe tener <code>setPubSubDomain(false)</code> .
Un solo técnico recibe todo .	Solo hay un contenedor de escucha activo, o los otros dos métodos no se detectaron.	Verificar que las tres anotaciones <code>@JmsListener</code> están en una clase con <code>@Component</code> dentro del paquete raíz.
NoUniqueBeanDefinitionException: expected single matching bean but found 2	Dos <code>JmsTemplate</code> y ningún <code>@Primary</code> ni <code>@Qualifier</code> .	Marcar uno con <code>@Primary</code> y cualificar en el punto de inyección.
client-id already added o Cannot create a subscriber on the durable subscription	Dos instancias de la aplicación con el mismo <code>clientId</code> , o dos oyentes compartiendo la fábrica durable.	Un <code>clientId</code> único por instancia; una sola suscripción durable por fábrica.
La suscripción durable no conserva nada.	Falta <code>clientId</code> , falta <code>subscriptionDurable(true)</code> o falta <code>subscription</code> .	Las tres condiciones deben cumplirse a la vez.

continúa

Síntoma o mensaje	Causa real	Solución
El mismo mensaje se procesa en bucle infinito.	El consumidor lanza siempre una excepción para ese mensaje.	Es el comportamiento esperado hasta agotar reintentos; revisar la configuración de la DLQ y hacer el consumidor idempotente.
El navegador dice Desconectado.	La ruta del canal SSE no coincide, o el tipo de contenido no es <code>text/event-stream</code> .	Comprobar que el <code>@GetMapping</code> declara <code>produces = TEXT_EVENT_STREAM_VALUE</code> y que el JavaScript apunta a <code>/api/flujo</code> .
El tablero solo se actualiza al recargar la página.	Se está usando <code>fetch</code> en bucle en lugar de <code>EventSource</code> .	Usar <code>EventSource</code> , como en el Listado 20.3.
Whitelabel Error Page al abrir <code>localhost:8080</code> .	Los archivos del frontend no están en <code>src/main/resources/static</code> .	Moverlos a esa ruta exacta y reiniciar.
Los cambios en el código no surten efecto.	DevTools no está activo o la compilación automática está deshabilitada.	Recompilar con <code>Ctrl+F9</code> o reiniciar la aplicación.

CAPÍTULO 24

Extensiones y aplicación a los proyectos del período

24.1 Cómo separar los componentes en procesos distintos

Todo lo construido vive en un proceso por comodidad didáctica. Separarlo es sorprendentemente barato, y ese es precisamente el argumento de la mensajería.

1. Crea un segundo proyecto Spring Boot con las mismas dependencias JMS y el mismo `application.properties`, salvo `server.port`, que debe ser distinto.
2. Copia `JmsConfig` y los `record` del dominio.
3. Lleva a ese proyecto uno de los suscriptores.
4. Arranca ambos a la vez.

No hay que cambiar ni una línea del productor. Ese es el resultado que conviene subrayar: el productor nunca supo, ni sabrá, dónde vive su audiencia.

El detalle que sí hay que cuidar al separar

Cuando emisor y receptor son proyectos distintos, sus `record` estarán en paquetes distintos, y el identificador de tipo que escribe el conversor dejará de coincidir. Hay que registrar un mapa de tipos en ambos lados:

```
Map<String, Class<?>> tipos = new HashMap<>();
tipos.put("evento-ticket", EventoTicket.class);
conversor.setTypeIdMappings(tipos);
```

Con ese mapa, el mensaje viaja con la etiqueta lógica "evento-ticket" y cada lado la traduce a su propia clase.

24.2 Traslado a los cuatro dominios del período

El mismo esqueleto se aplica a cualquiera de los proyectos vigentes. Lo único que cambia es qué mensaje es un comando y qué mensaje es un evento.

PFC	Cola: comando, un solo receptor	Tópico: evento, todos los interesados
AGLS TiendaTech	Reservar el inventario de un pedido: debe hacerlo un único proceso, o se vendería dos veces la misma unidad.	"Pedido confirmado": lo consumen facturación, notificación al comprador, analítica de ventas y el motor de recomendación.
BCEL SGA Escuela	Generar el certificado de matrícula en PDF: tarea pesada que debe ejecutarse una vez por solicitud.	"Nota registrada": lo consumen el cálculo de promedios, el aviso al representante y el registro académico.
FUVV Laboratorios	Confirmar una reserva de laboratorio: debe resolverla un único proceso para evitar reservas dobles.	"Laboratorio ocupado": lo consumen el tablero de disponibilidad, el control de acceso y las estadísticas de uso.
ACC Soporte ISP	Asignar el ticket a un técnico (el caso de este manual).	"Ticket cambió de estado": notificación, monitor de SLA y auditoría.

La prueba del algodón para decidir

Pregúntate qué ocurriría si *dos* procesos ejecutaran el mensaje. Si el resultado sería incorrecto —dos reservas, dos facturas, dos llamadas al cliente— es un comando y va a una cola. Si el resultado sería simplemente redundante o irrelevante, probablemente sea un evento y vaya a un tópico.

24.3 Relación con la calidad del producto

Las decisiones de este manual no son solo técnicas: cada una se corresponde con una característica del modelo de calidad de producto de la norma vigente [6].

Característica	Qué aporta la mensajería en este diseño
Fiabilidad	La cola conserva el trabajo si el consumidor cae; el reintento recupera fallos transitorios; la DLQ hace observables los fallos permanentes.
Eficiencia de desempeño	El hilo HTTP se libera de inmediato; añadir consumidores aumenta el rendimiento sin tocar código.
Mantenibilidad	Añadir un consumidor no modifica al emisor; el acoplamiento se reduce a un contrato de mensaje.
Compatibilidad	El uso de <code>TextMessage</code> con JSON permite que un consumidor escrito en otro lenguaje se integre sin cambios en el emisor.
Seguridad	Evitar <code>ObjectMessage</code> elimina un vector conocido de ejecución remota de código por deserialización.

24.4 Hacia dónde seguir

Tres extensiones naturales, en orden de dificultad creciente.

Persistencia e idempotencia.

Guardar los tickets en PostgreSQL y llevar un registro de identificadores ya procesados, de modo que una reentrega no duplique el trabajo. Es el paso imprescindible antes de llevar nada de esto a producción.

Transacciones y el patrón *outbox*.

Escribir en la base de datos y enviar el mensaje son dos operaciones sobre sistemas distintos; si una tiene éxito y la otra falla, el estado queda inconsistente. El patrón de bandeja de salida transaccional resuelve el problema sin recurrir a transacciones distribuidas, y está descrito con detalle en la literatura de referencia [7], [8].

Despliegue distribuido real.

Empaquetar cada componente en su propio contenedor y orquestarlos, momento en el que los patrones de composición de sistemas distribuidos cobran sentido pleno [1].

Una última observación sobre el catálogo de patrones

Prácticamente todo lo que aparece en este manual tiene nombre propio en el catálogo clásico de patrones de integración empresarial: *Point-to-Point Channel*, *Publish-Subscribe Channel*, *Competing Consumers*, *Message Selector*, *Dead Letter Channel*, *Durable Subscriber* [5]. Conocer esos nombres no es erudición: es lo que permite discutir un diseño con otro equipo sin tener que dibujarlo entero cada vez.

Bibliografía

- [1] B. Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Systems Using Kubernetes*, 2.^a ed. Sebastopol, CA, USA: O'Reilly Media, 2024, ISBN: 978-1-098-15635-0.
- [2] M. S. H. Chy, M. A. R. Arju, S. M. Tella y T. Cerny, "Comparative Evaluation of Java Virtual Machine-Based Message Queue Services: A Study on Kafka, Artemis, Pulsar, and RocketMQ," *Electronics*, vol. 12, n.º 23, pág. 4792, 2023. DOI: 10.3390/electronics12234792.
- [3] Eclipse Foundation, *Jakarta Messaging 3.1 Specification*, Eclipse Foundation, Brussels, Belgium, 2022. dirección: <https://jakarta.ee/specifications/messaging/3.1/> (visitado 30-07-2026).
- [4] P. T. Eugster, P. A. Felber, R. Guerraoui y A.-M. Kermarrec, "The Many Faces of Publish/Subscribe," *ACM Computing Surveys*, vol. 35, n.º 2, págs. 114-131, 2003. DOI: 10.1145/857076.857078.
- [5] G. Hohpe y B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA, USA: Addison-Wesley Professional, 2003, ISBN: 978-0-321-20068-6.
- [6] International Organization for Standardization, *ISO/IEC 25010:2023 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, 2.^a ed., ISO, Geneva, Switzerland, 2023.
- [7] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017, ISBN: 978-1-4493-7332-0.
- [8] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2.^a ed. Sebastopol, CA, USA: O'Reilly Media, 2021, ISBN: 978-1-492-03402-5.
- [9] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.^a ed. Maastricht, The Netherlands: Maarten van Steen, 2023, ISBN: 978-90-815406-3-6.